
ЧАСТЬ I

ОСНОВЫ

В этой части...

Глава 2. Переменные и базовые типы

Глава 3. Типы `string`, `vector` и массивы

Глава 4. Выражения

Глава 5. Операторы

Глава 6. Функции

Глава 7. Классы

Все широко распространенные языки программирования предоставляют единый набор средств, отличающийся лишь специфическими подробностями конкретного языка. Понимание подробностей того, как язык предоставляет эти средства, является первым шагом к овладению данным языком. К наиболее фундаментальным из этих общих средств относятся приведенные ниже.

- Встроенные типы данных (например, целые числа, символы и т.д.).
- Переменные, позволяющие присваивать имена используемым объектам.
- Выражения и операторы, позволяющие манипулировать значениями этих типов.
- Управляющие структуры, такие как `if` или `while`, обеспечивающие условное и циклическое выполнение наборов действий.
- Функции, позволяющие обратиться к именованным блокам действий.

Большинство языков программирования дополняет эти основные средства двумя способами: они позволяют программистам дополнять язык, определяя собственные типы, а также использовать библиотеки, в которых определены полезные функции и типы, отсутствующие в базовом языке.

В языке C++, как и в большинстве языков программирования, допустимые для объекта операции определяет его тип. То есть оператор будет допустимым или недопустимым в зависимости от типа используемого объекта. Некоторые языки, например Smalltalk и Python, проверяют используемые в выражениях

типы во время выполнения программы. В отличие от них, язык С++ осуществляет контроль типов данных статически, т.е. соответствие типов проверяется во время компиляции. Как следствие, компилятор требует сообщить ему тип каждого используемого в программе имени, прежде чем оно будет применено.

Язык С++ предоставляет набор встроенных типов данных, операторы для манипулирования ими и небольшой набор операторов для управления процессом выполнения программы. Эти элементы формируют алфавит, при помощи которого можно написать (и было написано) множество больших и сложных реальных систем. На этом базовом уровне язык С++ довольно прост. Его потрясающая мощь является результатом поддержки механизмов, которые позволяют программисту самостоятельно определять новые структуры данных. Используя эти средства, программисты могут приспособить язык для собственных целей без участия его разработчиков и необходимости ожидать, пока они удовлетворят появившиеся потребности.

Возможно, важнейшим компонентом языка С++ является класс, который позволяет программистам определять собственные типы данных. В языке С++ такие типы иногда называют “типами класса”, чтобы отличить их от базовых типов, встроенных в сам язык. Некоторые языки программирования позволяют определять типы, способные содержать только данные. Другие, подобно языку С++, позволяют определять типы, в состав которых можно включить операции, выполняемые с этими данными. Одна из главных задач проекта С++ заключалась в предоставлении программистам возможности самостоятельно определять типы данных, которые будут так же удобны, как и встроенные. Стандартная библиотека языка С++ использует эту возможность для реализации обширного набора классов и связанных с ними функций.

Первым шагом по овладению языком С++ является изучение его основ и библиотеки — такова тема части I, “Основы”. В главе 2 рассматриваются встроенные типы данных, а также обсуждается механизм определения новых, собственных типов. В главе 3 описаны два фундаментальных библиотечных типа: `string` (строка) и `vector` (вектор). В этой же главе рассматриваются массивы, представляющие собой низкоуровневую структуру данных, встроенную в язык С++, и множество других языков. Главы 4–6 посвящены выражениям, операторам и функциям. Завершается часть главой 7 демонстрирующей основы построения собственных типов классов. Как мы увидим, в определении собственных типов примиряется все, что мы изучили до сих пор, поскольку написание класса подразумевает использование всех средств, частично раскрытых в части I.

ГЛАВА 2

ПЕРЕМЕННЫЕ И БАЗОВЫЕ ТИПЫ

В этой главе...

2.1. Простые встроенные типы	61
2.2. Переменные	74
2.3. Составные типы	84
2.4. Спецификатор <code>const</code>	95
2.5. Работа с типами	105
2.6. Определение собственных структур данных	111
Резюме	118
Термины	118

Типы данных — это основа любой программы: они указывают, что именно означают эти данные и какие операции с ними можно выполнять.

У языка C++ обширная поддержка таких типов. В нем определено несколько базовых типов: символы, целые числа, числа с плавающей запятой и т.д. Язык предоставляет также механизмы, позволяющие программисту определять собственные типы данных. В библиотеке эти механизмы использованы для определения более сложных типов, таких как символьные строки переменной длины, векторы и т.д. В этой главе рассматриваются встроенные типы данных и основы применения более сложных типов.

Тип определяет назначение данных и операции, которые с ними можно выполнять. Например, назначение простого оператора `i = i + j`; полностью зависит от типов переменных `i` и `j`. Если это целые числа, данный оператор представляет собой обычное арифметическое сложение. Но если это объекты класса `Sales_item`, то данный оператор суммирует их компоненты (см раздел 1.5.1, стр. 47).

2.1. Простые встроенные типы

В языке C++ определен набор базовых типов, включая *арифметические типы* (arithmetic type), и специальный тип `void`. Арифметические типы пред-

ставляют символы, целые числа, логические значения и числа с плавающей запятой. С типом `void` не связано значений, и применяется он только при некоторых обстоятельствах, чаще всего как тип возвращаемого значения функций, которые не возвращают ничего.



2.1.1. Арифметические типы

Есть две разновидности арифметических типов: *целочисленные типы* (включая символьные и логические типы) и *типы с плавающей запятой*.

Размер (т.е. количество битов) арифметических типов зависит от конкретного компьютера. Стандарт гарантирует минимальные размеры, перечисленные в табл. 2.1. Однако компиляторы позволяют использовать для этих типов большие размеры. Поскольку количество битов не постоянно, значение одного типа также может занимать в памяти больше или меньше места.

Таблица 2.1. Арифметические типы языка C++

Тип	Значение	Минимальный размер
<code>bool</code>	Логический тип	Не определен
<code>char</code>	Символ	8 битов
<code>wchar_t</code>	Широкий символ	16 битов
<code>char16_t</code>	Символ Unicode	16 битов
<code>char32_t</code>	Символ Unicode	32 бита
<code>short</code>	Короткое целое число	16 битов
<code>int</code>	Целое число	16 битов
<code>long</code>	Длинное целое число	32 бита
<code>long long</code>	Длинное целое число	64 бита
<code>float</code>	Число с плавающей запятой одинарной точности	6 значащих цифр
<code>double</code>	Число с плавающей запятой двойной точности	10 значащих цифр
<code>long double</code>	Число с плавающей запятой повышенной точности	10 значащих цифр

Тип `bool` представляет только значения `true` (истина) и `false` (ложь).

Существует несколько символьных типов, большинство из которых предназначено для поддержки национальных наборов символов. Базовый символьный тип, `char`, гарантировано велик, чтобы содержать числовые значения, соответствующие символам базового набора символов машины. Таким образом, тип `char` имеет тот же размер, что и один байт на данной машине.

Остальные символьные типы, `wchar_t`, `char16_t` и `char32_t`, используются для расширенных наборов символов. Тип `wchar_t` будет достаточно большим, чтобы содержать любой символ в наибольшем расширенном наборе символов машины. Типы `char16_t` и `char32_t` предназначены для символов Unicode. (Unicode — это стандарт для представления символов, используемых, по существу, в любом языке.)

**C++
11**

Остальные целочисленные типы представляют целочисленные значения разных размеров. Язык C++ гарантирует, что тип `int` будет по крайней мере не меньше типа `short`, а тип `long` — не меньше типа `long`. Тип `long long` введен новым стандартом.

МАШИННЫЙ УРОВЕНЬ ПРЕДСТАВЛЕНИЯ ВСТРОЕННЫХ ТИПОВ

Компьютеры хранят данные как последовательность битов, каждый из которых содержит 0 или 1:

00011011011100010110010000111011 ...

Большинство компьютеров оперируют с памятью, разделенной на порции, размер которых в битах кратен степеням числа 2. Наименьшая порция адресуемой памяти называется *байтом* (byte). Основная единица хранения, обычно в несколько байтов, называется *словом* (word). В языке C++ байт содержит столько битов, сколько необходимо для содержания символа в базовом наборе символов машины. На большинстве компьютеров байт содержит 8 битов, а слово — 32 или 64 бита, т.е. 4 или 8 байтов.

У большинства компьютеров каждый байт памяти имеет номер, называемый *адресом* (address). На машине с 8-битовыми байтами и 32-битовыми словами слова в памяти можно было бы представить следующим образом:

736424	0	0	1	1	1	0	1	1
736425	0	0	0	1	1	0	1	1
736426	0	1	1	1	0	0	0	1
736427	0	1	1	0	0	1	0	0

Слева представлен адрес байта, а 8 битов его значения — справа.

При помощи адреса можно обратиться к любому из байтов, а также к набору из нескольких байтов, начинающемуся с этого адреса. В этом случае говорят о доступе к байту по адресу 736424 или о байте, хранящемся по адресу 736426. Чтобы получить представление о значении в области памяти по данному адресу, следует знать тип хранимого в ней значения. Именно тип определяет количество используемых битов и то, как эти биты интерпретировать.

Если известно, что объект в области по адресу 736424 имеет тип `float`, и если тип `float` на этой машине хранится в 32 битах, то известно и то, что объект по этому адресу охватывает все слово. Значение этого числа зависит от того, как именно машина хранит числа с плавающей запятой. Но если объект в области по адресу 736424 имеет тип `unsigned char`, то на машине, использующей набор символов ISO-Latin-1, этот байт представляет точку с запятой.

Типы с плавающей точкой представляют значения с одиночной, двойной и расширенной точностью. Стандарт определяет минимальное количество значащих цифр. Большинство компиляторов обеспечивает большую точность, чем минимально определено стандартом. Как правило, тип `float` представляется одним словом (32 бита), тип `double` — двумя словами (64 бита), а тип `long double` — тремя или четырьмя словами (96 или 128 битов). Типы `float` и `double` обычно имеют примерно по 7 и 16 значащих цифр соответственно. Тип `long double` зачастую используется для адаптации чисел с плавающей запятой аппаратных средств специального назначения; его точность, вероятно, также зависит от конкретной реализации этих средств.

Знаковые и беззнаковые типы

За исключением типа `bool` и расширенных символьных типов целочисленные типы могут быть *знаковыми* (`signed`) или *беззнаковыми* (`unsigned`). Знаковый тип способен представлять отрицательные и положительные числа (включая нуль); а беззнаковый тип — только положительные числа и нуль.

Типы `int`, `short`, `long` и `long long` являются знаковыми. Соответствующий беззнаковый тип получают добавлением части `unsigned` к названию такого типа, например `unsigned long`. Тип `unsigned int` может быть сокращен до `unsigned`.

В отличие от других целочисленных типов, существуют три разновидности базового типа `char`: `char`, `signed char` и `unsigned char`. В частности, тип `char` отличается от типа `signed char`. На три символьных типа есть только два представления: знаковый и беззнаковый. Простой тип `char` использует одно из этих представлений. Какое именно, зависит от компилятора.

В беззнаковом типе все биты представляют значение. Например, 8-битовый тип `unsigned char` может содержать значения от 0 до 255 включительно.

Стандарт не определяет представление знаковых типов, но он указывает, что диапазон должен быть поровну разделен между положительными и отрицательными значениями. Следовательно, 8-битовый тип `signed char` гарантированно будет в состоянии содержать значения от -127 до 127; большинство современных машин использует представления, позволяющие содержать значения от -128 до 127.

СОВЕТ. КАКОЙ ТИП ИСПОЛЬЗОВАТЬ

Подобно языку С, язык С++ был разработан так, чтобы по необходимости программа могла обращаться непосредственно к аппаратным средствам. Поэтому арифметические типы определены так, чтобы соответствовать особенностям различных аппаратных средств. В результате количество возможных арифметических типов в языке С++ огромно. Большинство программистов, желая избежать этих сложностей, ограничивают количество фактически используемых ими типов. Ниже приведено несколько эмпирических правил, способных помочь при выборе используемого типа.

- Используйте беззнаковый тип, когда точно знаете, что значения не могут быть отрицательными.
- Используйте тип `int` для целочисленной арифметики. Тип `short` обычно слишком мал, а тип `long` на практике зачастую имеет тот же размер, что и тип `int`. Если ваши значения больше, чем минимально гарантирует тип `int`, то используйте тип `long long`.
- Не используйте базовый тип `char` и тип `bool` в арифметических выражениях. Используйте их *только* для хранения символов и логических значений. Вычисления с использованием типа `char` особенно проблематичны, поскольку на одних машинах он знаковый, а на других беззнаковый. Если необходимо маленькое целое число, явно определите тип как `signed char` или `unsigned char`.
- Используйте тип `double` для вычислений с плавающей точкой. У типа `float` обычно недостаточно точности, а различие в затратах на вычисления с двойной и одинарной точностью незначительны. Фактически на некоторых машинах операции с двойной точностью осуществляются быстрее, чем с одинарной. Точность, предоставляемая типом `long double`, обычно чрезмерна и не нужна, а зачастую влечет значительное увеличение продолжительности выполнения.

Упражнения раздела 2.1.1

Упражнение 2.1. Каковы различия между типами `int`, `long`, `long long` и `short`? Между знаковыми и беззнаковыми типами? Между типами `float` и `double`?

Упражнение 2.2. Какие типы вы использовали бы для коэффициента, основной суммы и платежей при вычислении выплат по закладной? Объясните, почему вы выбрали каждый из типов?



2.1.2. Преобразование типов

Тип объекта определяет данные, которые он может содержать, и операции, которые с ним можно выполнять. Среди операций, поддерживаемых множеством типов, есть возможность *преобразовать* (convert) объект данного типа в другой, связанный тип.

Преобразование типов происходит автоматически, когда объект одного типа используется там, где ожидается объект другого типа. Более подробная информация о преобразованиях приведена в разделе 4.11 (стр. 218), а пока имеет смысл понять, что происходит при присвоении значения одного типа объекту другого.

Когда значение одного арифметического типа присваивается другому

```
bool b = 42;           // b содержит true
int i = b;             // i содержит значение 1
i = 3.14;              // i содержит значение 3
double pi = i;         // pi содержит значение 3.0
unsigned char c = -1;  // при 8-битовом char содержит значение 255
signed char c2 = 256;  // при 8-битовом char значение c2 неопределено
```

происходящее зависит от диапазона значений, поддерживаемых типом.

- Когда значение одного из не логических арифметических типов присваивается объекту типа `bool`, результат будет `false`, если значением является 0, а в противном случае — `true`.
- Когда значение типа `bool` присваивается одному из других арифметических типов, будет получено значение 1, если логическим значением было `true`, и 0, если это было `false`.
- Когда значение с плавающей точкой присваивается объекту целочисленного типа, оно усекается до части перед десятичной точкой.
- Когда целочисленное (интегральное) значение присваивается объекту типа с плавающей точкой, дробная часть равна нулю. Если у целого числа больше битов, чем может вместить объект с плавающей точкой, то точность может быть потеряна.
- Если объекту беззнакового типа присваивается значение не из его диапазона, результатом будет остаток от деления по модулю значения, которые способен содержать тип назначения. Например, 8-битовый тип `unsigned char` способен содержать значения от 0 до 255 включительно. Если присвоить ему значение вне этого диапазона, то компилятор присвоит ему остаток от деления по модулю 256. Поэтому в результате присвоения значения `-1` переменной 8-битового типа `unsigned char` будет получено значение 255.

- Если объекту знакового типа присваивается значение не из его диапазона, результат оказывается *неопределенен*. В результате программа может сработать нормально, а может и отказать или задействовать неверное значение.

СОВЕТ. ИЗБЕГАЙТЕ НЕОПРЕДЕЛЕННОГО И МАШИННО-ЗАВИСИМОГО ПОВЕДЕНИЯ

Результатом неопределенного поведения являются такие ошибки, которые компилятор не обязан (а иногда и не в состоянии) обнаруживать. Даже если код компилируется, то программа с неопределенным выражением все равно ошибочна.

К сожалению, программы, характеризующиеся неопределенным поведением на некоторых компиляторах и при некоторых обстоятельствах, могут работать вполне нормально, не проявляя проблему. Но нет никаких гарантий, что та же программа, откомпилированная на другом компиляторе или даже на следующей версии данного компилятора, продолжит работать правильно. Нет даже гарантий того, что, нормально работая с одним набором данных, она будет нормально работать с другим.

Аналогично в программах нельзя полагаться на машинно-зависимое поведение. Не стоит, например, надеяться на то, что переменная типа `int` имеет фиксированный, заранее известный размер. Такие программы называют *непереносимыми* (nonportable). При переносе такой программы на другую машину любой полагающийся на машинно-зависимое поведение код, вероятней всего, сработает неправильно, поэтому его придется искать и исправлять. Поиск подобных проблем в ранее нормально работавшей программе, мягко говоря, не самая приятная работа.

Компилятор применяет эти преобразования типов при использовании значений одного арифметического типа там, где ожидается значение другого арифметического типа. Например, при использовании значения, отличного от логического в условии, арифметическое значение преобразуется в тип `bool` таким же образом, как при присвоении арифметического значения переменной типа `bool`:

```
int i = 42;  
if (i) // условие рассматривается как истинное  
    i = 0;
```

При значении 0 условие будет ложным, а при всех остальных (отличных от нуля) — истинным.

К тому же при использовании значения типа `bool` в арифметическом выражении оно всегда преобразуется в 0 или 1. В результате применение логического значения в арифметическом выражении является неправильным.



Выражения, действующие беззнаковые типы

Хотя мы сами вряд ли преднамеренно присвоим отрицательное значение объекту беззнакового типа, мы можем (причем слишком легко) написать код, который сделает это неявно. Например, если использовать значения типа `unsigned` и `int` в арифметическом выражении, значения типа `int` обычно преобразуются в тип `unsigned`. Преобразование значения типа `int` в `unsigned` выполняется таким же способом, как и при присвоении:

```
unsigned u = 10;
int i = -42;
std::cout << i + i << std::endl; // выводит -84
std::cout << u + i << std::endl; // при 32-битовом int,
                                // выводит 4294967264
```

Во втором выражении, прежде чем будет осуществлено сложение, значение `-42` типа `int` преобразуется в значение типа `unsigned`. Преобразование отрицательного числа в тип `unsigned` происходит точно так же, как и при попытке присвоить это отрицательное значение объекту типа `unsigned`. Произойдет “обращение значения” (*wrap around*), как было описано выше.

При вычитании значения из беззнакового объекта, независимо от того, один или оба операнда являются беззнаковыми, следует быть уверенным том, что результат не окажется отрицательным:

```
unsigned u1 = 42, u2 = 10;
std::cout << u1 - u2 << std::endl; // ok: результат 32
std::cout << u2 - u1 << std::endl; // ok: но с обращением значения
```

Тот факт, что беззнаковый объект не может быть меньше нуля, влияет на способы написания циклов. Например, в упражнениях раздела 1.4.1 (стр. 39) следовало написать цикл, который использовал оператор декремента для вывода чисел от 10 до 0. Написанный вами цикл, вероятно, выглядел примерно так:

```
for (int i = 10; i >= 0; --i)
    std::cout << i << std::endl;
```

Казалось бы, этот цикл можно переписать, используя тип `unsigned`. В конце концов, мы не планируем выводить отрицательные числа. Однако это простое изменение типа приведет к тому, что цикл никогда не закончится:

```
// ОШИБКА: u никогда не сможет стать меньше 0; условие
// навсегда останется истинным
for (unsigned u = 10; u >= 0; --u)
    std::cout << u << std::endl;
```

Рассмотрим, что будет, когда `u` станет равно 0. На этой итерации отображается значение 0, а затем выполняется выражение цикла `for`. Это выражение, `--u`, вычитает 1 из `u`. Результат, `-1`, недопустим для беззнаковой переменной. Как и любое другое значение, не попадающее в диапазон допустимых, это будет преобразовано в беззнаковое значение. При 32-разрядном типе `int` результат выражения `--u` при `u` равном 0 составит 4294967295.

Исправить этот код можно, заменив цикл `for` циклом `while`, поскольку последний осуществляет декремент прежде (а не после) отображения значения:

```
unsigned u = 11; // начать цикл с элемента на один больше
               // первого, подлежащего отображению
while (u > 0) {
    --u; // сначала декремент, чтобы последняя итерация отобразила 0
    std::cout << u << std::endl;
}
```

Цикл начинается с декремента значения управляющей переменной цикла. В начале последней итерации переменная `u` будет иметь значение 1, а после декремента мы отобразим значение 0. При последующей проверке условия цикла `while` значением переменной `u` будет 0, и цикл завершится. Поскольку декремент осуществляется сначала, переменную `u` следует инициализировать значением на единицу больше первого подлежащего отображению значения. Следовательно, чтобы первым отображаемым значением было 10, переменную `u` инициализируем значением 11.

ВНИМАНИЕ! НЕ СМЕШИВАЙТЕ ЗНАКОВЫЕ И БЕЗЗНАКОВЫЕ ТИПЫ

Выражения, в которых смешаны знаковые и беззнаковые типы, могут приводить к удивительным результатам, когда знаковое значение оказывается негативным. Важно не забывать, что знаковые значения автоматически преобразовываются в беззнаковые. Например, в таком выражении, как `a * b`, если `a` содержит значение `-1`, а `b` значение 1 и обе переменные имеют тип `int`, ожидается результат `-1`. Но если переменная `a` имеет тип `int`, а переменная `b` — тип `unsigned`, то значение этого выражения будет зависеть от количества битов, занимаемых типом `int` на данной машине. На нашей машине результатом этого выражения оказалось 4294967295.

Упражнения раздела 2.1.2

Упражнение 2.3. Каков будет вывод следующего кода?

```
unsigned u = 10, u2 = 42;
std::cout << u2 - u << std::endl;
std::cout << u - u2 << std::endl;
int i = 10, i2 = 42;
```

```
std::cout << i2 - i << std::endl;
std::cout << i - i2 << std::endl;
std::cout << i - u << std::endl;
std::cout << u - i << std::endl;
```

Упражнение 2.4. Напишите программу для проверки правильности ответов. При неправильных ответах изучите этот раздел еще раз.

2.1.3. Литералы

Такое значение, как 42, в коде программы называется *литералом* (literal), поскольку его значение самоочевидно. У каждого литерала есть тип, определяемый его формой и значением.

Целочисленные литералы и литералы с плавающей запятой

Целочисленный литерал может быть в десятичной, восьмеричной или шестнадцатеричной форме. Целочисленные литералы, начинающиеся с нуля (0), интерпретируются как восьмеричные, а начинающиеся с 0x или 0X — как шестнадцатеричные. Например, значение 20 можно записать любым из трех следующих способов.

```
20      // десятичная форма
024     // восьмеричная форма
0x14    // шестнадцатеричная форма
```

Тип целочисленного литерала зависит от его значения и формы. По умолчанию десятичные литералы считаются знаковыми, а восьмеричные и шестнадцатеричные литералы могут быть знаковыми или беззнаковыми. Для десятичного литерала принимается наименьший тип, int, long, или long long, подходящий для его значения (т.е. первый подходящий в этом списке). Для восьмеричных и шестнадцатеричных литералов принимается наименьший тип, int, unsigned int, long, unsigned long, long long или unsigned long long, подходящий для значения литерала. Не следует использовать литерал, значение которого слишком велико для наибольшего соответствующего типа. Нет литералов типа short. Как можно заметить в табл. 2.2 (стр. 73), значения по умолчанию можно переопределить при помощи суффикса.

Хотя целочисленные литералы могут иметь знаковый тип, с технической точки зрения значение десятичного литерала никогда не бывает отрицательным числом. Если написать нечто, выглядящее как отрицательный десятичный литерал, например -42, то знак “минус” *не будет* частью литерала. Знак “минус” — это оператор, который инвертирует знак своего операнда (литерала).

Литералы с плавающей запятой включают либо десятичную точку, либо экспоненту, определенную при помощи экспоненциального представления. Экспонента в экспоненциальном представлении обозначается символом E или e:

```
3.14159    3.14159E0    0.    0e0    .001
```


По умолчанию литералы с плавающей запятой имеют тип `double`. Используя представленные в табл. 2.2 суффиксы, тип умолчанию можно переопределить.

Символьные и строковые литералы

Символ, заключенный в одинарные кавычки, является литералом типа `char`. Несколько символов, заключенных в парные кавычки, являются строковым литералом:

```
'a'           // символьный литерал
"Hello World!" // строковый литерал
```

Типом строкового литерала является массив константных символов. Этот тип обсуждается в разделе 3.5.4 (стр. 172). К каждому строковому литералу компилятор добавляет *нулевой символ* (null character) (`'\0'`). Таким образом, реальная величина строкового литерала на единицу больше его видимого размера. Например, литерал `'A'` представляет один символ `A`, тогда как строковый литерал `"A"` представляет массив из двух символов, символа `A` и нулевого символа.

Два строковых литерала, разделенных пробелами, табуляцией или символом новой строки, конкатенируются в единый литерал. Такую форму литерала используют, если необходимо написать слишком длинный текст, который неудобно располагать в одной строке.

```
// многострочный литерал
std::cout << "a really, really long string literal "
             "that spans two lines" << std::endl;
```

Управляющие последовательности

У некоторых символов, таких как возврат на один символ или управляющий символ, нет видимого изображения. Такие символы называют *непечатаемыми* (nonprintable character). Другие символы (одиночные и парные кавычки, вопросительный знак и наклонная черта влево) имеют в языке специальное назначение. В программах нельзя использовать ни один из этих символов непосредственно. Для их представления как символов используется *управляющая последовательность* (escape sequence), начинающаяся с символа наклонной черты влево.

В языке C++ определены следующие управляющие последовательности.

Новая строка (newline)	<code>\n</code>	Горизонтальная табуляция (horizontal tab)	<code>\t</code>	Оповещение, звонок (alert)	<code>\a</code>
Вертикальная табуляция (vertical tab)	<code>\v</code>	Возврат на один символ (backspace)	<code>\b</code>	Двойная кавычка (double quote)	<code>\"</code>
Наклонная черта влево (backslash)	<code>\\</code>	Вопросительный знак (question mark)	<code>\?</code>	Одинарная кавычка (single quote)	<code>\'</code>
Возврат каретки (carriage return)	<code>\r</code>	Прогон страницы (formfeed)	<code>\f</code>		

Управляющую последовательность используют как единый символ:

```
std::cout << '\n';           // отобразить новую строку
std::cout << "\tHi!\n";      // отобразить табуляцию,
                             // текст "Hi!" и новую строку
```

Можно также написать обобщенную управляющую последовательность, где за `\x` следует одна или несколько шестнадцатеричных цифр или за `\` следует одна, две или три восьмеричные цифры. Так можно отобразить символ по его числовому значению. Вот несколько примеров (подразумевается использование набора символов Latin-1):

```
\7 (оповещение)           \12 (новая строка)       \40 (пробел)
\0 (нулевой символ)       \115 (символ 'М')       \x4d (символ 'М')
```

Как и управляющие последовательности, определенные языком, такой синтаксис можно использовать вместо любого другого символа:

```
std::cout << "Hi \x4d0\115!\n"; // выводит Hi MOM! и новую строку
std::cout << '\115' << '\n';    // выводит М и новую строку
```

Обратите внимание: если символ `\` сопровождается более чем тремя восьмеричными цифрами, то ассоциируются с ним только первые три. Например, литерал `"\1234"` представляет два символа: символ, представленный восьмеричным значением 123, и символ 4. Форма `\x`, напротив, использует все последующие шестнадцатеричные цифры; литерал `"\x1234"` представляет один 16-разрядный символ, состоящий из битов, соответствующих этим четырем шестнадцатеричным цифрам. Поскольку большинство машин использует 8-битовые символы, подобные значения вряд ли будут полезны. Обычно шестнадцатеричные символы с более чем 8 битами используются для расширенных наборов символов с применением одного из префиксов, приведенных в табл. 2.2.

Определение типа литерала

При помощи суффикса или префикса, представленного в табл. 2.2, можно переопределить заданный по умолчанию тип целого числа, числа с плавающей запятой или символьного литерала.

```
L'a'           // литерал типа wchar_t (широкий символ)
u8"hi!"        // строковый литерал utf-8 (8-битовая кодировка Unicode)
42ULL          // целочисленный беззнаковый литерал, тип unsigned long long
1E-3F          // литерал с плавающей точкой и одинарной точностью, тип float
3.14159L       // литерал с плавающей точкой и расширенной точностью,
                // тип long double
```

При обозначении литерала как имеющего тип `long` используйте букву `L` в верхнем регистре; строчная буква `l` слишком похожа на цифру 1.

Таблица 2.2. Определение типа литерала

Символьные и строковые литералы			
Префикс	Значение	Тип	
U	Символ Unicode 16	char16_t	
U	Символ Unicode 32	char32_t	
L	Широкий символ	wchar_t	
U8	utf-8 (только строковые литералы)	Char	

Целочисленные литералы		Литералы с плавающей точкой	
Суффикс	Минимальный тип	Суффикс	Тип
u или U	Unsigned	f или F	float
l или L	Long	l или L	long double
Ll или LL	long long		

Можно непосредственно определить знак и размер целочисленного литерала. Если суффикс содержит символ U, то у литерала беззнаковый тип. Таким образом, у десятичного, восьмеричного или шестнадцатеричного литерала с суффиксом U будет наименьший тип unsigned int, unsigned long или unsigned long long, в соответствии со значением литерала. Если суффикс будет содержать символ L, то типом литерала будет по крайней мере long; если суффикс будет содержать символы LL, то типом литерала будет long long или unsigned long long.

Можно объединить символ U с символом L или символами LL. Литерал с суффиксом UL, например, задаст тип unsigned long или unsigned long long, в зависимости от того, помещается ли его значение в тип unsigned long.

Логические литералы и литеральные указатели

Слова true и false — это логические литералы (литералы типа bool).

```
bool test = false;
```

Слово nullptr является литеральным указателем. Более подробная информация об указателях и литерале nullptr приведена в разделе 2.3.2 (стр. 86).

Упражнения раздела 2.1.3

Упражнение 2.5. Определите тип каждого из следующих литералов. Объясните различия между ними:

- (a) 'a', L'a', "a", L"a"
- (b) 10, 10u, 10L, 10uL, 012, 0xC
- (c) 3.14, 3.14f, 3.14L
- (d) 10, 10u, 10., 10e-2

Упражнение 2.6. Имеются ли различия между следующими определениями:

```
int month = 9, day = 7;
int month = 09, day = 07;
```

Упражнение 2.7. Какие значения представляют эти литералы? Какой тип имеет каждый из них?

- (a) "Who goes with F\145rgus?\012"
- (b) 3.14e1L (c) 1024f (d) 3.14L

Упражнение 2.8. Напишите программу, использующую управляющие последовательности для вывода значения 2M, сопровождаемого новой строкой. Модифицируйте программу так, чтобы вывести 2, затем табуляцию, потом M и наконец символ новой строки.

2.2. Переменные

Переменная (variable) — это именованное хранилище, которым могут манипулировать программы. У каждой переменной в языке C++ есть тип. Тип определяет размер и расположение переменной в памяти, диапазон значений, которые могут храниться в ней, и набор применимых к переменной операций. Программисты C++ используют термины “переменная” и “объект” как синонимы.



2.2.1. Определения переменных

Простое определение переменной состоит из *спецификатора типа* (type specifier), сопровождаемого списком из одного или нескольких имен переменных, отделенных запятыми, и завершающей точки с запятой. Тип каждого имени в списке задан спецификатором типа. Определение может (не обязательно) предоставить исходное значение для одного или нескольких определяемых имен:

```
int sum = 0, value, // sum, value и units_sold имеют тип int
    units_sold = 0; // sum и units_sold инициализированы значением 0
Sales_item item;    // item имеет тип Sales_item (см. p. 1.5.1 (стр. 47))
// string — библиотечный тип, представляющий последовательность символов
// переменной длины
std::string book("0-201-78345-X"); // book инициализирована строковым
// литералом
```

В определении переменной `book` использован библиотечный тип `std::string`. Подобно классу `iostream` (см. раздел 1.2 стр. 30), класс `string` определен в пространстве имен `std`. Более подробная информация о классе `string` приведена в главе 3, а пока достаточно знать то, что тип `string` представляет последовательность символов переменной длины. Библиотечный тип `string` предоставляет несколько способов инициализации строковых объектов. Один из них — копирование строкового литерала (см. раздел 2.1.3 стр. 70). Таким образом, переменная `book` инициализируется символами `0-201-78345-X`.

ТЕРМИНОЛОГИЯ. ЧТО ТАКОЕ ОБЪЕКТ?

Программисты языка C++ используют термин *объект* (object) часто, и не всегда по делу. В самом общем определении объект — это область памяти, способная содержать данные и обладающая типом.

Одни программисты используют термин *объект* лишь для переменных и экземпляров классов. Другие используют его, чтобы различать именованные и неименованные объекты, причем для именованных объектов используют термин *переменная* (variable). Третьи различают объекты и значения, используя термин *объект* для тех данных, которые могут быть изменены программой, и термин *значение* (value) — для тех данных, которые предназначены только для чтения.

В этой книге используется наиболее общий смысл термина *объект*, т.е. область памяти, для которой указан тип. Здесь под объектом подразумеваются практически все используемые в программе данные, независимо от того, имеют ли они встроенный тип или тип класса, являются ли они именованными или нет, предназначены только для чтения или допускают изменение.

Инициализаторы

Инициализация (initialization) присваивает объекту определенное значение в момент его создания. Используемые для инициализации переменных значения могут быть насколько угодно сложными выражениями. Когда определяется несколько переменных, имена всех объектов следуют непосредственно друг за другом. Таким образом, вполне возможно инициализировать переменную значением одной из переменных, определенных ранее в том же определении.

```
// ok: переменная price определяется и инициализируется прежде,  
// чем она будет использована для инициализации переменной discount  
double price = 109.99, discount = price * 0.16;  
// ok: Вызов функции applyDiscount() и использование ее возвращаемого  
// значения для инициализации переменной salePrice  
salePrice = applyDiscount(price, discount);
```

Инициализация в C++ — на удивление сложная тема, и мы еще не раз вернемся к ней. Многих программистов вводит в заблуждение использование символа `=` при инициализации переменной. Они полагают, что инициализация — это такая форма присвоения, но в C++ инициализация и присвоение — совершенно разные операции. Эта концепция особенно важна, поскольку во многих языках это различие несущественно и может быть проигнорировано. Тем не менее даже в языке C++ это различие зачастую не имеет значения. Однако данная концепция крайне важна, и мы будем повторять это еще не раз.



Инициализация — это не присвоение. Инициализация переменной происходит при ее создании. Присвоение удаляет текущее значение объекта и заменяет его новым.

Списочная инициализация

Тема инициализации настолько сложна потому, что язык поддерживает ее в нескольких разных формах. Например, для определения переменной `units_sold` типа `int` и ее инициализации значением `0` можно использовать любой из следующих четырех способов:

```
int units_sold = 0;
int units_sold = {0};
int units_sold{0};
int units_sold(0);
```



Использование фигурных скобок для инициализации было введено новым стандартом. Ранее эта форма инициализации допускалась лишь в некоторых случаях. По причинам, описанным в разделе 3.3.1 (стр. 142), эта форма инициализации известна как *списочная инициализация* (list initialization). Списки инициализаторов в скобках можно теперь использовать всегда, когда инициализируется объект, и в некоторых случаях, когда объекту присваивается новое значение.

При использовании с переменными встроенного типа эта форма инициализации обладает важным преимуществом: компилятор не позволит инициализировать переменные встроенного типа, если инициализатор может привести к потере информации:

```
long double ld = 3.1415926536;
int a{ld}, b = {ld}; // ошибка: преобразование с потерей
int c(ld), d = ld;   // ok: но значение будет усечено
```

Компилятор откажет в инициализации переменных `a` и `b`, поскольку использование значения типа `long double` для инициализации переменной типа `int` может привести к потере данных. Как минимум, дробная часть значения переменной `ld` будет усечена. Кроме того, целочисленная часть значения

переменной `ld` может быть слишком большой, чтобы поместиться в переменную типа `int`.

То, что здесь представлено, может показаться тривиальным, в конце концов, вряд ли кто инициализирует переменную типа `int` значением типа `long double` непосредственно. Однако, как представлено в главе 16, такая инициализация может произойти непреднамеренно. Более подробная информация об этих формах инициализации приведена в разделах 3.2.1 (стр. 126) и 3.3.1 (стр. 142).

Инициализация по умолчанию

При определении переменной без инициализатора происходит ее *инициализация по умолчанию* (default initialization). Таким переменным присваивается *значение по умолчанию* (default value). Это значение зависит от типа переменной и может также зависеть от того, где определяется переменная.

Значение объекта встроенного типа, не инициализированного явно, зависит от того, где именно он определяется. Переменные, определенные вне тела функции, инициализируются значением 0. За одним рассматриваемым вскоре исключением, определенные в функции переменные встроенного типа остаются *неинициализированными* (uninitialized). Значение неинициализированной переменной встроенного типа неопределенно (см. раздел 2.1.2, стр. 67). Попытка копирования или получения доступа к значению неинициализированной переменной является ошибкой.

Инициализацию объекта типа класса контролирует сам класс. В частности, класс позволяет определить, могут ли быть созданы его объекты без инициализатора. Если это возможно, класс определяет значение, которое будет иметь его объект в таком случае.

Большинство классов позволяет определять объекты без явных инициализаторов. Такие классы самостоятельно предоставляют соответствующее значение по умолчанию. Например, новый объект библиотечного класса `string` без инициализатора является пустой строкой.

```
std::string empty; // неявно инициализируется пустой строкой
Sales_item item;   // объект Sales_item инициализируется
                  // значением по умолчанию
```

Однако некоторые классы требуют, чтобы каждый объект был инициализирован явно. При попытке создать объект такого класса без инициализатора компилятор пожалуется на это.



Значение неинициализированных объектов встроенного типа, определенных в теле функции, неопределенно. Значение не инициализируемых явно объектов типа класса определяется классом.

Упражнения раздела 2.2.1

Упражнение 2.9. Объясните следующие определения. Если среди них есть некорректные, объясните, что не так и как это исправить.

- (a) `std::cin >> int input_value;` (b) `int i = { 3.14 };`
(c) `double salary = wage = 9999.99;` (d) `int i = 3.14;`

Упражнение 2.10. Каковы исходные значения, если таковые вообще имеются, каждой из следующих переменных?

```
std::string global_str;  
int global_int;  
int main()  
{  
    int local_int;  
    std::string local_str;  
}
```



2.2.2. Объявления и определения переменных

Для обеспечения возможности разделить программу на несколько логических частей язык C++ предоставляет технологию, известную как *раздельная компиляция* (separate compilation). Раздельная компиляция позволяет составлять программу из нескольких файлов, каждый из которых может быть откомпилирован независимо.

При разделении программы на несколько файлов необходим способ совместного использования кода этих файлов. Например, код, определенный в одном файле, возможно, должен использовать переменную, определенную в другом файле. В качестве конкретного примера рассмотрим объекты `std::cout` и `std::cin`. Классы этих объектов определены где-то в стандартной библиотеке, но все же наши программы могут использовать их.

ВНИМАНИЕ! НЕИНИЦИАЛИЗИРОВАННЫЕ ПЕРЕМЕННЫЕ — ПРИЧИНА ПРОБЛЕМ ВО ВРЕМЯ ВЫПОЛНЕНИЯ

Значение неинициализированной переменной неопределенно. Попытка использования значения неинициализированной переменной является ошибкой, которую зачастую трудно обнаружить. Кроме того, компилятор не обязан обнаруживать такие ошибки, хотя большинство из них предупреждает, по крайней мере, о некоторых случаях использования неинициализированных переменных.

Что же произойдет при использовании неинициализированной переменной с неопределенным значением? Иногда (если повезет) программа отказывается сразу, при попытке доступа к объекту. Обнаружив место, где происходит отказ, как правило, довольно просто выяснить, что его причи-

ной является неправильно инициализированная переменная. Но иногда программа срабатывает, хотя результат получается ошибочным. Возможен даже худший вариант, когда на одной машине результаты получаются правильными, а на другой происходит сбой. Кроме того, добавление кода во вполне работоспособную программу в неподходящем месте тоже может привести к внезапному возникновению проблем.



Мы рекомендуем инициализировать каждый объект встроенного типа. Это не всегда необходимо, но проще и безопасней предоставить инициализатор, чем выяснять, можно ли в данном конкретном случае безопасно опустить его.

Для поддержки раздельной компиляции язык C++ различает объявления и определения. *Объявление* (declaration) делает имя известным программе. Файл, который должен использовать имя, определенное в другом месте, включает объявление для этого имени. *Определение* (definition) создает соответствующую сущность.

Объявление переменной определяет ее тип и имя. Определение переменной — это ее объявление. Кроме задания имени и типа, определение резервирует также место для ее хранения и может снабдить переменную исходным значением.

Чтобы получить объявление, не являющееся также определением, добавляется ключевое слово `extern` и можно не предоставлять явный инициализатор.

```
extern int i; // объявить, но не определить переменную i
int j;       // объявить и определить переменную j
```

Любое объявление, которое включает явный инициализатор, является определением. Для переменной, определенной как `extern` (внешняя), можно предоставить инициализатор, но это отменит ее определение как `extern`. Объявление внешней переменной с инициализатором является ее определением:

```
extern double pi = 3.1416; // определение
```

Предоставление инициализатора внешней переменной в функции является ошибкой.



Объявлены переменные могут быть много раз, но определены только однажды.

На настоящий момент различие между объявлением и определением может показаться неочевидным, но фактически оно очень важно. Использование переменной в нескольких файлах требует объявлений, отдельных от определения. Чтобы использовать ту же переменную в нескольких файлах, ее следует

определить в одном, и только одном файле. В других файлах, где используется та же переменная, ее следует объявить, но не определять.

Более подробная информация о том, как язык C++ поддерживает раздельную компиляцию, приведена в разделах 2.6.3 (стр. 116) и 6.1.3 (стр. 274).

Упражнения раздела 2.2.2

Упражнение 2.11. Объясните, приведены ли ниже объявления или определения.

- (a) `extern int ix = 1024;`
- (b) `int iy;`
- (c) `extern int iz;`

Ключевая концепция. Статическая типизация

Язык C++ обладает строгим *статическим контролем типов* (statically typed) данных. Это значит, что проверка соответствия значений заявленным для них типам данных осуществляется во время компиляции. Сам процесс проверки называют *контролем соответствия типов* (type-checking), или *типизацией* (typing).

Как уже упоминалось, тип ограничивает операции, которые можно выполнять с объектом. В языке C++ компилятор проверяет, поддерживает ли используемый тип операции, которые с ним выполняют. Если обнаруживается попытка сделать нечто, не поддерживаемое данным типом, компилятор выдает сообщение об ошибке и не создает исполняемый файл.

По мере усложнения рассматриваемых программ будет со всей очевидностью продемонстрировано, что строгий контроль соответствия типов способен помочь при поиске ошибок в исходном коде. Однако последствием статической проверки является то, что тип каждой используемой сущности должен быть известен компилятору. Следовательно, тип переменной необходимо объявить прежде, чем эту переменную можно будет использовать.

2.2.3. Идентификаторы

Идентификаторы (identifier) (или имена) в языке C++ могут состоять из символов, цифр и символов подчеркивания. Язык не налагает ограничений на длину имен. Идентификаторы должны начинаться с букв или символа подчеркивания. Символы в верхнем и нижнем регистрах различаются, т.е. идентификаторы языка C++ чувствительны к регистру.

```
// определено четыре разных переменных типа int
int somename, someName, SomeName, SOMENAME;
```

Язык резервирует набор имен, перечисленных в табл. 2.3 и 2.4, для собственных нужд. Эти имена не могут использоваться как идентификаторы.

Таблица 2.3. Ключевые слова языка C++

<code>alignas</code>	<code>continue</code>	<code>friend</code>	<code>register</code>	<code>true</code>
<code>alignof</code>	<code>decltype</code>	<code>goto</code>	<code>reinterpret_cast</code>	<code>try</code>
<code>asm</code>	<code>default</code>	<code>if</code>	<code>return</code>	<code>typedef</code>
<code>auto</code>	<code>delete</code>	<code>inline</code>	<code>short</code>	<code>typeid</code>
<code>bool</code>	<code>do</code>	<code>int</code>	<code>signed</code>	<code>typename</code>
<code>break</code>	<code>double</code>	<code>long</code>	<code>sizeof</code>	<code>union</code>
<code>case</code>	<code>dynamic_cast</code>	<code>mutable</code>	<code>static</code>	<code>unsigned</code>
<code>catch</code>	<code>else</code>	<code>namespace</code>	<code>static_assert</code>	<code>using</code>
<code>char</code>	<code>enum</code>	<code>new</code>	<code>static_cast</code>	<code>virtual</code>
<code>char16_t</code>	<code>explicit</code>	<code>noexcept</code>	<code>struct</code>	<code>void</code>
<code>char32_t</code>	<code>export</code>	<code>nullptr</code>	<code>switch</code>	<code>volatile</code>
<code>class</code>	<code>extern</code>	<code>operator</code>	<code>template</code>	<code>wchar_t</code>
<code>const</code>	<code>false</code>	<code>private</code>	<code>this</code>	<code>while</code>
<code>constexpr</code>	<code>float</code>	<code>protected</code>	<code>thread_local</code>	
<code>const_cast</code>	<code>for</code>	<code>public</code>	<code>throw</code>	

Таблица 2.4. Альтернативные имена операторов языка C++

<code>and</code>	<code>bitand</code>	<code>compl</code>	<code>not_eq</code>	<code>or_eq</code>	<code>xor_eq</code>
<code>and_eq</code>	<code>bitor</code>	<code>not</code>	<code>or</code>	<code>xor</code>	

Кроме ключевых слов, стандарт резервирует также набор идентификаторов для использования в библиотеке, поэтому пользовательские идентификаторы не могут содержать два последовательных символа подчеркивания, а также начинаться с символа подчеркивания, непосредственно за которым следует прописная буква. Кроме того, идентификаторы, определенные вне функций, не могут начинаться с символа подчеркивания.

Соглашения об именах переменных

Существует множество общепринятых соглашений для именования переменных. Применение подобных соглашений может существенно улучшать удобочитаемость кода.

- Идентификатор должен быть осмысленным.
- Имена переменных обычно состоят из строчных символов. Например, `index`, а не `Index` или `INDEX`.

- Имена классов обычно начинаются с прописной буквы, например `Sales_item`.
- Несколько слов в идентификаторе разделяют либо символом подчеркивания, либо прописными буквами в первых символах каждого слова. Например: `student_loan` или `studentLoan`, но не `studentloan`.


 Рекомендуем

Самым важным аспектом соглашения об именовании является его неукоснительное соблюдение.

Упражнения раздела 2.2.3

Упражнение 2.12. Какие из приведенных ниже имен недопустимы (если таковые есть)?

- (a) `int double = 3.14;` (b) `int _;`
 (c) `int catch-22;` (d) `int 1_or_2 = 1;`
 (e) `double Double = 3.14;`

2.2.4. Область видимости имен

В любом месте программы каждое используемое имя относится к вполне определенной сущности — переменной, функции, типу и т.д. Однако имя может быть использовано многократно для обращения к различным сущностям в разных точках программы.

Область видимости (scope) — это часть программы, в которой у имени есть конкретное значение. Как правило, области видимости в языке C++ разграничиваются фигурными скобками.

В разных областях видимости то же имя может относиться к разным сущностям. Имена видимы от момента их объявления и до конца области видимости, в которой они объявлены.

В качестве примера рассмотрим программу из раздела 1.4.2 (стр. 40):

```
#include <iostream>
int main()
{
    int sum = 0;
    // сложить числа от 1 до 10 включительно
    for (int val = 1; val <= 10; ++val)
        sum += val; // эквивалентно sum = sum + val
    std::cout << "Sum of 1 to 10 inclusive is "
              << sum << std::endl;
    return 0;
}
```

Эта программа определяет три имени — `main`, `sum` и `val`, а также использует имя пространства имен `std`, наряду с двумя именами из этого пространства имен — `cout` и `endl`.

Имя `main` определено вне фигурных скобок. Оно, как и большинство имен, определенных вне функции, имеет глобальную область видимости (*global score*). Будучи объявлены, имена в глобальной области видимости доступны в программе повсюду. Имя `sum` определено в пределах блока, которым является тело функции `main()`. Оно доступно от момента объявления и далее в остальной части функции `main()`, но не за ее пределами. Переменная `sum` имеет область видимости блока (*block score*). Имя `val` определяется в пределах оператора `for`. Оно применимо только в этом операторе, но не в другом месте функции `main()`.

СОВЕТ. ОПРЕДЕЛЯЙТЕ ПЕРЕМЕННЫЕ ПРИ ПЕРВОМ ИСПОЛЬЗОВАНИИ

Объект имеет смысл определять поближе к месту его первого использования. Это улучшает удобочитаемость и облегчает поиск определения переменной. Однако важнее всего то, что когда переменная определяется ближе к месту ее первого использования, зачастую проще присвоить ей подходящее исходное значение.

Вложенные области видимости

Области видимости могут содержать другие области видимости. Содержащая (или вложенная) область видимости называется *внутренней областью видимости* (*inner score*), а содержащая ее области видимости — *внешней областью видимости* (*outer score*).

Как только имя объявлено в области видимости, оно становится доступно во вложенных в нее областях видимости. Имена, объявленные во внешней области видимости, могут быть также переопределены во внутренней области видимости:

```
#include <iostream>
// Программа предназначена исключительно для демонстрации.
// Использование в функции глобальной переменной, а также определение
// одноименной локальной переменной - это очень плохой стиль
// программирования
int reused = 42; // reused имеет глобальную область видимости
int main()
{
    int unique = 0; // unique имеет область видимости блока
    // вывод #1: используется глобальная reused; выводит 42 0
    std::cout << reused << " " << unique << std::endl;
    int reused = 0; // новый локальный объект по имени reused скрывает
                    // глобальный reused
    // вывод #2: используется локальная reused; выводит 0 0
    std::cout << reused << " " << unique << std::endl;
    // вывод #3: явное обращение к глобальной reused; выводит 42 0
    std::cout << ::reused << " " << unique << std::endl;
    return 0;
}
```

Вывод #1 осуществляется перед определением локальной переменной `reused`. Поэтому данный оператор вывода использует имя `reused`, определенное в глобальной области видимости. Этот оператор выводит 42 0. Вывод #2 происходит после определения локальной переменной `reused`. Теперь локальная переменная `reused` находится в области видимости (*in scope*). Таким образом, второй оператор вывода использует локальный объект `reused`, а не глобальный и выводит 0 0. Вывод #3 использует оператор области видимости (см. раздел 1.2, стр. 33) для переопределения стандартных правил областей видимости. У глобальной области видимости нет имени. Следовательно, когда у оператора области видимости пусто слева, это обращение к указанному справа имени в глобальной области видимости. Таким образом, это выражение использует глобальный объект `reused` и выводит 42 0.



ВНИМАНИЕ

Как правило, определение локальных переменных, имена которых совпадают с именами глобальных переменных, является крайне неудачным решением.

Упражнения раздела 2.2.4

Упражнение 2.13. Каково значение переменной `j` в следующей программе?

```
int i = 42;
int main()
{
    int i = 100;
    int j = i;
}
```

Упражнение 2.14. Допустим ли следующий код? Если да, то какие значения он отобразит на экране?

```
int i = 100, sum = 0;
for (int i = 0; i != 10; ++i)
    sum += i;
std::cout << i << " " << sum << std::endl;
```



2.3. Составные типы

Составной тип (compound type) — это тип, определенный в терминах другого типа. У языка C++ есть несколько составных типов, два из которых, ссылки и указатели, мы рассмотрим в этой главе.

У рассмотренных на настоящий момент объявлений не было ничего, кроме имен переменных. Такие переменные имели простейший, базовый тип объявления. Более сложные операторы объявления позволяют определять переменные с составными типами, которые состоят из объявлений базового типа.



2.3.1. Ссылки

Ссылка (reference) является альтернативным именем объекта. Ссылочный тип “ссылается на” другой тип. В определении ссылочного типа используется оператор объявления в форме `&d`, где `d` — объявляемое имя:

```
int ival = 1024;
int &refVal = ival; // refVal ссылается на другое имя, ival
int &refVal2;      // ошибка: ссылку следует инициализировать
```

Обычно при инициализации переменной значение инициализатора копируется в создаваемый объект. При определении ссылки вместо копирования значения инициализатора происходит *связывание* (bind) ссылки с ее инициализатором. После инициализации ссылка остается связанной с исходным объектом. Нет никакого способа изменить привязку ссылки так, чтобы она ссылалась на другой объект, поэтому ссылки *следует* инициализировать.



Новый стандарт ввел новый вид ссылки — *ссылка r-значения* (r-value reference), которую мы рассмотрим в разделе 13.6.1 (стр. 673). Эти ссылки предназначены прежде всего для использования в классах. С технической точки зрения, когда мы используем термин *ссылка* (reference), мы подразумеваем *ссылку l-значения* (l-value reference).



Ссылка — это псевдоним

После того как ссылка определена, *все* операции с ней фактически осуществляются с объектом, с которым связана ссылка.

```
refVal = 2;          // присваивает значение 2 объекту, на который ссылается
                    // ссылка refVal, т.е. ival
int ii = refVal;     // то же, что и ii = ival
```



Ссылка — это не объект, а *только другое имя уже существующего объекта*.

При присвоении ссылки присваивается объект, с которым она связана. При доступе к значению ссылки фактически происходит обращение к значению объекта, с которым связана ссылка. Точно так же, когда ссылка используется как инициализатор, в действительности для этого используется объект, с которым связана ссылка.

```
// ok: ссылка refVal3 связывается с объектом, с которым связана
// ссылка refVal, т.е. с ival
int &refVal3 = refVal;
// инициализирует i значением объекта, с которым связана ссылка refVal
int i = refVal; // ok: инициализирует i значением ival
```

Поскольку ссылки не объекты, нельзя определить ссылку на ссылку.

Определение ссылок

В одном определении можно определить несколько ссылок. Каждому являющемуся ссылкой идентификатору должен предшествовать символ `&`.

```
int i = 1024, i2 = 2048; // i и i2 — переменные типа int
int &r = i, r2 = i2;     // r — ссылка, связанная с переменной i;
                        // r2 — переменная типа int
int i3 = 1024, &ri = i3; // i3 — переменная типа int;
                        // ri — ссылка, связанная с переменной i3
int &r3 = i3, &r4 = i2;  // r3 и r4 — ссылки
```

За двумя исключениями, рассматриваемыми в разделах 2.4.1 (стр. 98) и 15.2.3 (стр. 759), типы ссылки и объекта, на который она ссылается, должны совпадать точно. Кроме того, по причинам, рассматриваемым в разделе 2.4.1, ссылка может быть связана только с объектом, но не с литералом или результатом более общего выражения:

```
int &refVal4 = 10;      // ошибка: инициализатор должен быть объектом
double dval = 3.14;
int &refVal5 = dval;    // ошибка: инициализатор должен быть объектом
                        // типа int
```

Упражнения раздела 2.3.1

Упражнение 2.15. Какие из следующих определений недопустимы (если таковые есть)? Почему?

```
(a) int ival = 1.01;      (b) int &rval1 = 1.01;
(c) int &rval2 = ival;    (d) int &rval3;
```

Упражнение 2.16. Какие из следующих присвоений недопустимы (если таковые есть)? Если они допустимы, объясните, что они делают.

```
int i = 0, &r1 = i; double d = 0, &r2 = d;
(a) r2 = 3.14159;      (b) r2 = r1;
(c) i = r2;             (d) r1 = d;
```

Упражнение 2.17. Что выводит следующий код?

```
int i, &ri = i;
i = 5; ri = 10;
std::cout << i << " " << ri << std::endl;
```



2.3.2. Указатели

Указатель (pointer) — это составной тип, переменная которого указывает на объект другого типа. Подобно ссылкам, указатели используются для косвенного доступа к другим объектам. В отличие от ссылок, указатель — это настоящий объект. Указатели могут быть присвоены и скопированы; один указатель за время своего существования может указывать на несколько разных объектов. В отличие от ссылки, указатель можно не инициализировать в момент определения. Подоб-

но объектам других встроенных типов, значение неинициализированного указателя, определенного в области видимости блока, неопределенно.



ВНИМАНИЕ

Указатели зачастую трудно понять. При отладке проблемы, связанные с ошибками в указателях, способны запутать даже опытных программистов.

Тип указателя определяется оператором в форме `*d`, где `d` — определяемое имя. Символ `*` следует повторять для каждой переменной указателя.

```
int *ip1, *ip2; // ip1 и ip2 — указатели на тип int
double dp, *dp2; // dp2 — указатель на тип double;
                // dp — переменная типа double
```

Получение адреса объекта

Указатель содержит адрес другого объекта. Для получения адреса объекта используется *оператор обращения к адресу* (address-of operator), или *оператор &*.

```
int ival = 42;
int *p = &ival; // p содержит адрес переменной ival;
                // p — указатель на переменную ival
```

Второй оператор определяет `p` как указатель на тип `int` и инициализирует его адресом объекта `ival` типа `int`. Поскольку ссылки не объекты, у них нет адресов, а следовательно, невозможно определить указатель на ссылку.

За двумя исключениями, рассматриваемыми в разделах 2.4.2 (стр. 99) и 15.2.3 (стр. 759), типы указателя и объекта, на который он указывает, должны совпадать.

```
double dval;
double *pd = &dval; // ok: инициализатор — адрес объекта типа double
double *pd2 = pd;   // ok: инициализатор — указатель на тип double
int *pi = pd;       // ошибка: типы pi и pd отличаются
pi = &dval;         // ошибка: присвоение адреса типа double
                    // указателю на тип int
```

Типы должны совпадать, поскольку тип указателя используется для вывода типа объекта, на который он указывает. Если бы указатель содержал адрес объекта другого типа, то выполнение операций с основным объектом потерпело бы неудачу.

Значение указателя

Хранимое в указателе значение (т.е. адрес) может находиться в одном из четырех состояний.

1. Оно может указывать на объект.
2. Оно может указывать на область непосредственно за концом объекта.

3. Это может быть нулевое значение, означающее, что данный указатель не связан ни с одним объектом.
4. Оно может быть недопустимо. Любое иное значение, кроме приведенного выше, является недопустимым.

Копирование или иная попытка доступа к значению по недопустимому указателю является серьезной ошибкой. Как и использование неинициализированной переменной, компилятор вряд ли обнаружит эту ошибку. Результат доступа к недопустимому указателю непредсказуем. Поэтому всегда следует знать, допустим ли данный указатель.

Хотя указатели в случаях 2 и 3 допустимы, действия с ними ограничены. Поскольку эти указатели не указывают ни на какой объект, их нельзя использовать для доступа к объекту. Если все же сделать попытку доступа к объекту по такому указателю, то результат будет непредсказуем.

Использование указателя для доступа к объекту

Когда указатель указывает на объект, для доступа к этому объекту можно использовать *оператор обращения к значению* (dereference operator), или *оператор **.

```
int ival = 42;
int *p = &ival; // p содержит адрес ival; p - указатель на ival
cout << *p;     // * возвращает объект, на который указывает p;
                // выводит 42
```

Обращение к значению указателя возвращает объект, на который указывает указатель. Присвоив значение результату оператора обращения к значению, можно присвоить его самому объекту.

```
*p = 0;          // * возвращает объект; присвоение нового значения
                // ival через указатель p
cout << *p;      // выводит 0
```

При присвоении значения **p* оно присваивается объекту, на который указывает указатель *p*.



Обратиться к значению можно только по допустимому указателю, который указывает на объект.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. У НЕКОТОРЫХ СИМВОЛОВ ЕСТЬ НЕСКОЛЬКО ЗНАЧЕНИЙ

Некоторые символы, такие как *&* и ***, используются и как оператор в выражении, и как часть объявления. Контекст, в котором используется символ, определяет то, что он означает.

```
int i = 42;
int &r = i; // & следует за типом в части объявления; r - ссылка
```

```

int *p;          // * следует за типом в части объявления; p - указатель
p = &i;          // & используется в выражении как оператор
                  // обращения к адресу
*p = i;          // * используется в выражении как оператор
                  // обращения к значению
int &r2 = *p;     // & в части объявления; * - оператор обращения к значению

```

В объявлениях символы `&` и `*` используются для формирования составных типов. В выражениях эти же символы используются для обозначения оператора. Поскольку тот же символ используется в совершенно ином смысле, возможно, стоит игнорировать внешнее сходство и считать их как будто различными символами.

Нулевые указатели

Нулевой указатель (null pointer) не указывает ни на какой объект. Код может проверить, не является ли указатель нулевым, прежде чем пытаться использовать его. Есть несколько способов получить нулевой указатель.

```

int *p1 = nullptr; // эквивалентно int *p1 = 0;
int *p2 = 0;        // непосредственно инициализирует p2 литеральной
                    // константой 0, необходимо #include cstdlib
int *p3 = NULL;     // эквивалентно int *p3 = 0;

```

**C++
11**

Проще всего инициализировать указатель, используя *литерал* `nullptr`, который был введен новым стандартом. Литерал `nullptr` имеет специальный тип, который может быть преобразован (см. раздел 2.1.2, стр. 66) в любой другой ссылочный тип. В качестве альтернативы можно инициализировать указатель литералом `0`, как это сделано в определении указателя `p2`.

Программисты со стажем иногда используют *переменную препроцессора* (preprocessor variable) `NULL`, которую заголовок `cstdlib` определяет как `0`.

Немного подробнее препроцессор рассматривается в разделе 2.6.3 (стр. 116), а пока достаточно знать, что *препроцессор* (preprocessor) — это программа, которая выполняется перед компилятором. Переменные препроцессора используются препроцессором, они не являются частью пространства имен `std`, поэтому их указывают непосредственно, без префикса `std::`.

При использовании переменной препроцессора последний автоматически заменяет такую переменную ее значением. Следовательно, инициализация указателя переменной `NULL` эквивалентна его инициализации значением `0`. Сейчас программы C++ вообще должны избегать применения переменной `NULL` и использовать вместо нее литерал `nullptr`.

Нельзя присваивать переменную типа `int` указателю, даже если ее значением является `0`.

```

int zero = 0;
pi = zero; // ошибка: нельзя присвоить переменную типа int указателю

```

СОВЕТ. ИНИЦИАЛИЗИРУЙТЕ ВСЕ УКАЗАТЕЛИ

Неинициализированные указатели — обычный источник ошибок времени выполнения.

Подобно любой другой неинициализированной переменной, последствия использования неинициализированного указателя непредсказуемы. Использование неинициализированного указателя почти всегда приводит к аварийному отказу во время выполнения. Однако поиск причин таких отказов может оказаться на удивление трудным.

У большинства компиляторов при использовании неинициализированного указателя биты в памяти, где он располагается, используются как адрес. Использование неинициализированного указателя — это попытка доступа к несуществующему объекту в произвольной области памяти. Нет никакого способа отличить допустимый адрес от недопустимого, состоящего из случайных битов, находящихся в той области памяти, которая была зарезервирована для указателя.

Авторы рекомендуют инициализировать все переменные, а особенно указатели. Если это возможно, определяйте указатель только после определения объекта, на который он должен указывать. Если связываемого с указателем объекта еще нет, то инициализируйте указатель значением `nullptr` или 0. Так код программы может узнать, что указатель не указывает на объект.

Присвоение и указатели

И указатели, и ссылки предоставляют косвенный доступ к другим объектам. Однако есть важные различия в способе, которым они это делают. Самое важное то, что ссылка — это не объект. После того как ссылка определена, нет никакого способа заставить ее ссылаться на другой объект. При использовании ссылки всегда используется объект, с которым она была связана первоначально.

Между указателем и содержащимся в нем адресом нет такой связи. Подобно любой другой (нессылочной) переменной, при присвоении указателя для него устанавливается новое значение. Присвоение заставляет указатель указывать на другой объект.

```
int i = 42;
int *pi = 0;    // указатель pi инициализирован, но не адресом объекта
int *pi2 = &i;  // указатель pi2 инициализирован адресом объекта i
int *pi3;       // если pi3 определен в блоке, pi3 не инициализирован
pi3 = pi2;      // pi3 и pi2 указывают на тот же объект, т.е. на i
pi2 = 0;        // теперь pi2 не содержит адреса никакого объекта
```

Сначала может быть трудно понять, изменяет ли присвоение указатель или сам объект, на который он указывает. Важно не забывать, что присвоение из-

меняет свой левый операнд. Следующий код присваивает новое значение переменной `pi`, что изменяет адрес, который она хранит:

```
pi = &ival; // значение pi изменено; теперь pi указывает на ival
```

С другой стороны, следующий код (использующий `*pi`, т.е. значение, на которое указывает указатель `pi`) изменяет значение объекта:

```
*pi = 0; // значение ival изменено; pi неизменен
```

Другие операции с указателями

Пока значение указателя допустимо, его можно использовать в условии. Аналогично использованию арифметических значений (раздел 2.1.2 стр. 66), если указатель содержит значение 0, то условие считается ложным.

```
int ival = 1024;
int *pi = 0;      // pi допустим, нулевой указатель
int *pi2 = &ival; // pi2 допустим, содержит адрес ival
if (pi)           // pi содержит значение 0, условие считается ложным
    // ...
if (pi2)          // pi2 указывает на ival, значит, содержит не 0;
    // условие считается истинным
    // ...
```

Любой отличный от нулевого указатель рассматривается как значение `true`.

Два допустимых указателя того же типа можно сравнить, используя операторы равенства (`==`) и неравенства (`!=`). Результат этих операторов имеет тип `bool`. Два указателя равны, если они содержат одинаковый адрес, и неравны в противном случае. Два указателя содержат одинаковый адрес (т.е. равны), если они оба нулевые, если они указывают на тот же объект или на область непосредственно за концом того же объекта. Обратите внимание, что указатель на объект и указатель на область за концом другого объекта вполне могут содержать одинаковый адрес. Такие указатели равны.

Поскольку операции сравнения используют значения указателей, эти указатели должны быть допустимы. Результат использования недопустимого указателя в условии или в сравнении непредсказуем.

Дополнительные операции с указателями будут описаны в разделе 3.5.3 (стр. 167).

Тип `void*` является специальным типом указателя, способного содержать адрес любого объекта. Подобно любому другому указателю, указатель `void*` содержит адрес, но тип объекта по этому адресу неизвестен.

```
double obj = 3.14, *pd = &obj;
// ok: void* может содержать адрес любого типа данных
void *pv = &obj; // obj может быть объектом любого типа
pv = pd;         // pv может содержать указатель на любой тип
```

С указателем `void*` допустимо немного действий: его можно сравнить с другим указателем, можно передать его функции или возвратить из нее либо присвоить другому указателю типа `void*`. Его нельзя использовать для работы с объектом, адрес которого он содержит, поскольку неизвестен тип объекта, неизвестны и операции, которые можно с ним выполнять.

Как правило, указатель `void*` используют для работы с памятью как с областью памяти, а не для доступа к объекту, хранящемуся в этой области. Использование указателей `void*` рассматривается в разделе 19.1.1 (стр. 1025), а в разделе 4.11.3 (стр. 224) продемонстрировано, как можно получить адрес, хранящийся в указателе `void*`.

Упражнения раздела 2.3.2

Упражнение 2.18. Напишите код, изменяющий значение указателя. Напишите код для изменения значения, на которое указывает указатель.

Упражнение 2.19. Объясните основные отличия между указателями и ссылками.

Упражнение 2.20. Что делает следующая программа?

```
int i = 42;
int *p1 = &i;
*p1 = *p1 * *p1;
```

Упражнение 2.21. Объясните каждое из следующих определений. Укажите, все ли они корректны и почему.

```
int i = 0;
(a) double* dp = &i; (b) int *ip = i; (c) int *p = &i;
```

Упражнение 2.22. С учетом того, что `p` является указателем на тип `int`, объясните следующий код:

```
if (p) // ...
if (*p) // ...
```

Упражнение 2.23. Есть указатель `p`, можно ли определить, указывает ли он на допустимый объект? Если да, то как? Если нет, то почему?

Упражнение 2.24. Почему инициализация указателя `p` допустима, а указателя `lp` нет?

```
int i = 42;      void *p = &i;      long *lp = &i;
```



2.3.3. Понятие описаний составных типов

Как уже упоминалось, определение переменной состоит из указания базового типа и списка операторов объявления. Каждый оператор объявления может связать свою переменную с базовым типом отлично от других операторов объявления в том же определении. Таким образом, одно определение может определять переменные различных типов.

```
// i - переменная типа int; p - указатель на тип int;
// r - ссылка на тип int
int i = 1024, *p = &i, &r = i;
```



Многие программисты не понимают взаимодействия базового и модифицированного типа, который может быть частью оператора объявления.



Определение нескольких переменных

Весьма распространенное заблуждение полагать, что модификатор типа (* или &) применяется ко всем переменным, определенным в одном операторе. Частично причина в том, что между модификатором типа и объявляемым именем может находиться пробел.

```
int* p; // вполне допустимо, но может ввести в заблуждение
```

Данное определение может ввести в заблуждение потому, что создается впечатление, будто `int*` является типом каждой переменной, объявленной в этом операторе. Несмотря на внешний вид, базовым типом этого объявления является `int`, а не `int*`. Символ `*` — это модификатор типа `p`, он не имеет никакого отношения к любым другим объектам, которые могли бы быть объявлены в том же операторе:

```
int* p1, p2; // p1 - указатель на тип int; p2 - переменная типа int
```

Есть два общепринятых стиля определения нескольких переменных с типом указателя или ссылки. Согласно первому, модификатор типа располагается рядом с идентификатором:

```
int *p1, *p2; // p1 и p2 - указатели на тип int
```

Этот стиль подчеркивает, что переменная имеет составной тип.

Согласно второму, модификатор типа располагается рядом с типом, но он определяет только одну переменную в операторе:

```
int* p1; // p1 - указатель на тип int
int* p2; // p2 - указатель на тип int
```

Этот стиль подчеркивает, что объявление определяет составной тип.



Нет никакого единственно правильного способа определения указателей и ссылок. Важно неукоснительно придерживаться выбранного стиля.

В этой книге используется первый стиль, знак `*` (или `&`) помещается рядом с именем переменной.

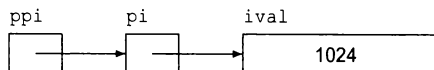
Указатели на указатели

Теоретически нет предела количеству модификаторов типа, применяемых в операторе объявления. Когда модификаторов более одного, они объединяются хоть и логичным, но не всегда очевидным способом. В качестве примера рассмотрим указатель. Указатель — это объект в памяти, и, как у любого объекта, у этого есть адрес. Поэтому можно сохранить адрес указателя в другом указателе.

Каждый уровень указателя отмечается собственным символом *. Таким образом, для указателя на указатель пишут **, для указателя на указатель на указатель — *** и т.д.

```
int ival = 1024;
int *pi = &ival; // pi указывает на переменную типа int
int **ppi = &pi; // ppi указывает на указатель на переменную типа int
```

Здесь `pi` — указатель на переменную типа `int`, а `ppi` — указатель на указатель на переменную типа. Эти объекты можно было бы представить так:



Подобно тому, как обращение к значению указателя на переменную типа `int` возвращает значение типа `int`, обращение к значению указателя на указатель возвращает указатель. Для доступа к основному объекту в этом случае необходимо обратиться к значению указателя дважды:

```
cout << "The value of ival\n"
    << "direct value: " << ival << "\n"
    << "indirect value: " << *pi << "\n"
    << "doubly indirect value: " << **ppi
    << endl;
```

Эта программа выводит значение переменной `ival` тремя разными способами: сначала непосредственно, затем через указатель `pi` на тип `int` и наконец обращением к значению указателя `ppi` дважды, чтобы добраться до основного значения в переменной `ival`.

Ссылки на указатели

Ссылка — не объект. Следовательно, не может быть указателя на ссылку. Но поскольку указатель — это объект, вполне можно определить ссылку на указатель.

```
int i = 42;
int *p; // p - указатель на тип int
int *r = p; // r - ссылка на указатель p
r = &i; // r ссылается на указатель;
// присвоение &i ссылке r делает p указателем на i
*r = 0; // обращение к значению r дает i, объект, на который
// указывает p; изменяет значение i на 0
```


Проще всего понять тип `r` — прочитать определение справа налево. Ближайший символ к имени переменной (в данном случае `&` в `&r`) непосредственно влияет на тип переменной. Таким образом, становится ясно, что `r` является ссылкой. Остальная часть оператора объявления определяет тип, на который ссылается ссылка `r`. Следующий символ, в данном случае `*`, указывает, что тип `r` относится к типу указателя. И наконец, базовый тип объявления указывает, что `r` — это ссылка на указатель на переменную типа `int`.



Сложное объявление указателя или ссылки может быть проще понять, если читать его справа налево.

Упражнения раздела 2.3.3

Упражнение 2.25. Определите типы и значения каждой из следующих переменных:

(a) `int* ip, &r = ip;` (b) `int i, *ip = 0;` (c) `int* ip, ip2;`



2.4. Спецификатор `const`

Иногда необходимо определить переменную, значение которой, как известно, не может быть изменено. Например, можно было бы использовать переменную, хранящую размер буфера. Использование переменной облегчит изменение размера буфера, если мы решим, что исходный размер нас не устраивает. С другой стороны, желательно предотвратить непреднамеренное изменение в коде значения этой переменной. Значение этой переменной можно сделать неизменным, используя в ее определении *спецификатор* `const` (qualifier `const`):

```
const int bufSize = 512; // размер буфера ввода
```

Это определит переменную `bufSize` как константу. Любая попытка присвоить ей значение будет ошибкой:

```
bufSize = 512; // ошибка: попытка записи в константный объект
```

Поскольку нельзя изменить значение константного объекта после создания, его следует инициализировать. Как обычно, инициализатор может быть выражением любой сложности:

```
const int i = get_size(); // ok: инициализация во время выполнения
const int j = 42;         // ok: инициализация во время компиляции
const int k;              // ошибка: k - неинициализированная константа
```

Инициализация и константы

Как уже упоминалось не раз, тип объекта определяет операции, которые можно с ним выполнять. Константный тип можно использовать для большинства, но не для всех операций, как и его неконстантный аналог. Ограничение одно — можно использовать только те операции, которые неспособны изменить объект. Например, тип `const int` можно использовать в арифметических выражениях точно так же, как обычный неконстантный тип `int`. Тип `const int` преобразуется в тип `bool` тем же способом, что и обычный тип `int`, и т.д.

К операциям, не изменяющим значение объекта, относится инициализация. При использовании объекта для инициализации другого объекта не имеет значения, один или оба из них являются константами.

```
int i = 42;
const int ci = i; // ok: значение i копируется в ci
int j = ci;      // ok: значение ci копируется в j
```

Хотя переменная `ci` имеет тип `const int`, ее значение имеет тип `int`. Константность переменной `ci` имеет значение только для операций, которые могли бы изменить ее значение. При копировании переменной `ci` для инициализации переменной `j` ее константность не имеет значения. Копирование объекта не изменяет его. Как только копия сделана, у нового объекта нет никакой дальнейшей связи с исходным объектом.

По умолчанию константные объекты локальны для файла

Когда константный объект инициализируется константой во время компиляции, такой как `bufSize` в определении ниже, компилятор обычно заменяет используемую переменную ее значением во время компиляции.

```
const int bufSize = 512; // размер буфера ввода
```

Таким образом, компилятор создаст исполняемый код, использующий значение 512 в тех местах, где исходный код использует переменную `bufSize`.

Чтобы заменить переменную значением, компилятор должен видеть ее инициализатор. При разделении программы на несколько файлов, в каждом из которых используется константа, необходим доступ к ее инициализатору. Для этого переменная должна быть определена в каждом файле, в котором используется ее значение (см. раздел 2.2.2, стр. 79). Для обеспечения такого поведения, но все же без повторных определений той же переменной, константные переменные определяются как локальные для файла. Определение константы с тем же именем в нескольких файлах подобно написанию определения для отдельных переменных в каждом файле.

Иногда константу необходимо совместно использовать в нескольких файлах, однако ее инициализатор не является константным выражением. Мы не

хотим, чтобы компилятор создал отдельную переменную в каждом файле, константный объект должен вести себя как другие (не константные) переменные. В таком случае определить константу следует в одном файле, и объявить ее в других файлах, где она тоже используется.

Для определения единого экземпляра константной переменной используется ключевое слово `extern` как в ее определении, так и в ее объявлениях.

```
// Файл file_1.cc. Определение и инициализация константы, которая
// доступна для других файлов
extern const int bufSize = fcn();
// Файл file_1.h
extern const int bufSize; // та же bufSize, определенная в file_1.cc
```

Здесь переменная `bufSize` определяется и инициализируется в файле `file_1.cc`. Поскольку это объявление включает инициализатор, оно (как обычно) является и определением. Но поскольку `bufSize` константа, необходимо применить ключевое слово `extern`, чтобы использовать ее в других файлах.

Объявление в заголовке `file_1.h` также использует ключевое слово `extern`. В данном случае это демонстрирует, что имя `bufSize` не является локальным для этого файла и что его определение находится в другом месте.



Чтобы совместно использовать константный объект в нескольких файлах, его необходимо определить с использованием ключевого слова `extern`.

Упражнения раздела 2.4

Упражнение 2.26. Что из приведенного ниже допустимо? Если что-то недопустимо, то почему?

- (a) `const int buf;` (b) `int cnt = 0;`
 (c) `const int sz = cnt;` (d) `++cnt; ++sz;`



2.4.1. Ссылка на константу

Подобно любым другим объектам, с константным объектом можно связать ссылку. Для этого используется *ссылка на константу* (reference to const), т.е. ссылка на объект типа `const`. В отличие от обычной ссылки, ссылку на константу нельзя использовать для изменения объекта, с которым она связана.

```
const int ci = 1024;
const int &r1 = ci; // ok: и ссылка, и основной объект - константы
r1 = 42;           // ошибка: r1 - ссылка на константу
int &r2 = ci;       // ошибка: неконстантная ссылка на константный объект
```

Поскольку нельзя присвоить значение самой переменной `ci`, ссылка также не должна позволять изменять ее. Поэтому инициализация ссылки `r2` — это ошибка. Если бы эта инициализация была допустима, то ссылке `r2` можно было бы использовать для изменения значения ее основного объекта.

ТЕРМИНОЛОГИЯ. КОНСТАНТНАЯ ССЫЛКА — ЭТО ССЫЛКА НА КОНСТАНТУ

Программисты C++, как правило, используют термин *константная ссылка* (*const reference*), однако фактически речь идет о *ссылке на константу* (*reference to const*).

С технической точки зрения нет никаких константных ссылок. Ссылка — не объект, поэтому саму ссылку нельзя сделать константой. На самом деле, поскольку нет никакого способа заставить ссылку ссылаться на другой объект, то в некотором смысле все ссылки — константы. То, что ссылка ссылается на константный или неконстантный тип, относится к тому, что при помощи этой ссылки можно сделать, однако привязку самой ссылки изменить нельзя в любом случае.

Инициализация и ссылки на константу

В разделе 2.1.2 (стр. 86) мы обращали ваше внимание на два исключения из правила, согласно которому тип ссылки должен совпадать с типом объекта, на который она ссылается. Первое исключение: мы можем инициализировать ссылку на константу результатом выражения, тип которого может быть преобразован (см. раздел 2.1.2 стр. 66) в тип ссылки. В частности, мы можем связать ссылку на константу с неконстантным объектом, литералом или более общим выражением:

```
int i = 42;
const int &r1 = i;           // можно связать ссылку const int& с обычным
                             // объектом int
const int &r2 = 42;         // ok: r1 - ссылка на константу
const int &r3 = r1 * 2;     // ok: r3 - ссылка на константу
int &r4 = r * 2;           // ошибка: r4 - простая, неконстантная ссылка
```

Простейший способ понять это различие в правилах инициализации — рассмотреть то, что происходит при связывании ссылки с объектом другого типа:

```
double dval = 3.14;
const int &ri = dval;
```

Здесь ссылка `ri` ссылается на переменную типа `int`. Операции со ссылкой `ri` будут целочисленными, но переменная `dval` содержит число с плавающей запятой, а не целое число. Чтобы удостовериться в том, что объект, с которым связана ссылка `ri`, имеет тип `int`, компилятор преобразует этот код в нечто следующее:

```
const int temp = dval; // создать временную константу типа int из
                      // переменной типа double
const int &ri = temp;  // связать ссылку ri с временной константой
```

В данном случае ссылка `ri` связана с *временным объектом* (temporary). Временный объект — это безымянный объект, создаваемый компилятором для хранения промежуточного результата вычисления. Программисты C++ зачастую используют слово “temporary” как сокращение термина “temporary object”.

Теперь рассмотрим, что могло бы произойти, будь инициализация позволена, но ссылка `ri` не была бы константной. В этом случае мы могли бы присвоить значение по ссылке `ri`. Это изменило бы объект, с которым связана ссылка `ri`. Этот временный объект имеет тип не `dval`. Программист, заставивший ссылку `ri` ссылаться на переменную `dval`, вероятно, ожидал, что присвоение по ссылке `ri` изменит переменную `dval`. В конце концов, почему произошло присвоение по ссылке `ri`, если не было намерения изменить объект, с которым она связана? Поскольку связь ссылки с временным объектом осуществляет уж конечно *не то*, что подразумевал программист, язык считает это некорректным.

Ссылка на константу может ссылаться на неконстантный объект

Важно понимать, что ссылка на константу ограничивает только то, что при помощи этой ссылки можно делать. Привязка ссылки к константному объекту ничего не говорит о том, является ли сам основной объект константой. Поскольку основной объект может оказаться неконстантным, он может быть изменен другими способами:

```
int i = 42;
int &r1 = i;          // r1 связана с i
const int &r2 = i;    // r2 тоже связана с i;
                    // но она не может использоваться для изменения i
r1 = 0;              // r1 - неконстантна; i теперь 0
r2 = 0;              // ошибка: r2 - ссылка на константу
```

Привязка ссылки `r2` к неконстантной переменной `i` типа `int` вполне допустима. Но ссылке `r2` нельзя использовать для изменения значения переменной `i`. Несмотря на это, значение переменной `i` вполне можно изменить другим способом. Например, можно присвоить ей значение непосредственно или при помощи другой связанной с ней ссылки, такой как `r1`.



2.4.2. Указатели и спецификатор `const`

Подобно ссылкам, вполне возможно определять указатели, которые указывают на объект константного или неконстантного типа. Как и ссылку на константу (см. раздел 2.4.1, стр. 97), *указатель на константу* (pointer to const) невозможно использовать для изменения объекта, на который он указывает. Адрес константного объекта можно хранить только в указателе на константу:

```
const double pi = 3.14;    // pi - константа; ее значение неизменно
double *ptr = &pi;        // ошибка: ptr - простой указатель
const double *cptr = &pi; // ok: cptr может указывать на тип const double
*cptr = 42;               // ошибка: нельзя присвоить *cptr
```

В разделе 2.3.2 (стр. 86) упоминалось о наличии двух исключений из правила, согласно которому типы указателя и объекта, на который он указывает, должны совпадать. Первое исключение — это возможность использования указателя на константу для указания на неконстантный объект:

```
double dval = 3.14; // dval типа double; ее значение неизменно
cptr = &dval;       // ok: но изменить dval при помощи cptr нельзя
```

Подобно ссылке на константу, указатель на константу ничего не говорит о том, является ли объект, на который он указывает, константой. Определение указателя как указателя на константу влияет только на то, что с его помощью можно сделать. Не забывайте, что нет никакой гарантии того, что объект, на который указывает указатель на константу, не будет изменяться.



Возможно, указатели и ссылки на константы следует рассматривать как указатели или ссылки, "которые полагают, что они указывают или ссылаются на константы".

Константные указатели

В отличие от ссылок, указатели — это объекты. Следовательно, подобно любым другим объектам, вполне может быть указатель, сам являющийся константой. Как и любой другой константный объект, константный указатель следует инициализировать, после чего изменить его значение (т.е. адрес, который он содержит) больше нельзя. Константный указатель объявляют, расположив ключевое слово `const` после символа `*`. Это означает, что данный указатель является константой, а не обычным указателем на константу.

```
int errNumb = 0;
int *const curErr = &errNumb; // curErr всегда будет указывать на errNumb
const double pi = 3.14159;
const double *const pip = &pi; // pip константный указатель на
                               // константный объект
```

Как уже упоминалось в разделе 2.3.3 (стр. 95), проще всего понять эти объявления, читая их справа налево. В данном случае ближе всего к имени `curErr` расположен спецификатор `const`, означая, что сам объект `curErr` будет константным. Тип этого объекта формирует остальная часть оператора объявления. Следующий символ оператора объявления, `*`, означает, что `curErr` — это константный указатель. И наконец, объявление завершает базовый тип, означая, что `curErr` — это константный указатель на объект типа `int`. Аналогично `pip` — это константный указатель на объект типа `const double`.

Тот факт, что указатель сам является константой, ничто не говорит о том, можем ли мы использовать указатель для изменения основного объекта. Возможность изменения объекта полностью зависит от типа, на который указывает указатель. Например, `pip` — это константный указатель на константу. Ни значение объекта, на который указывает указатель `pip`, ни хранящийся в нем адрес не могут быть изменены. С другой стороны, указатель `curErr` имеет простой, неконстантный тип `int`. Указатель `curErr` можно использовать для изменения значения переменной `errNumb`:

```
*pip = 2.72; // ошибка: pip - указатель на константу
// если значение объекта, на который указывает указатель curErr
// (т.е. errNumb), отлично от нуля
if (*curErr) {
    errorHandler();
    *curErr = 0; // обнулить значение объекта, на который
                // указывает указатель curErr
}
```

Упражнения раздела 2.4.2

Упражнение 2.27. Какие из следующих инициализаций допустимы? Объясните почему.

- (a) `int i = -1, &r = 0;` (b) `int *const p2 = &i2;`
 (c) `const int i = -1, &r = 0;` (d) `const int *const p3 = &i2;`
 (e) `const int *p1 = &i2;` (f) `const int &const r2;`
 (g) `const int i2 = i, &r = i;`

Упражнение 2.28. Объясните следующие определения. Какие из них недопустимы?

- (a) `int i, *const cp;` (b) `int *p1, *const p2;`
 (c) `const int ic, &r = ic;` (d) `const int *const p3;`
 (e) `const int *p;`

Упражнение 2.29. С учетом переменных из предыдущих упражнений, какие из следующих присвоений допустимы? Объясните почему.

- (a) `i = ic;` (b) `p1 = p3;`
 (c) `p1 = ⁣` (d) `p3 = ⁣`
 (e) `p2 = p1;` (f) `ic = *p3;`



2.4.3. Спецификатор const верхнего уровня

Как уже упоминалось, указатель — это объект, способный указывать на другой объект. В результате можно сразу сказать, является ли указатель сам константой и являются ли константой объекты, на которые он может указывать. Термин *спецификатор const верхнего уровня* (top-level const) используется для обозначения того ключевого слова `const`, которое объявляет константой сам указатель. Когда указатель способен указывать на константный объект, это называется *спецификатор const нижнего уровня* (low-level const).

В более общем смысле спецификатор `const` верхнего уровня означает, что объект сам константа. Спецификатор `const` верхнего уровня может присутствовать в любом типе объекта, будь то один из встроенных арифметических типов, тип класса или ссылочный тип. Спецификатор `const` нижнего уровня присутствует в базовом типе составных типов, таких как указатели или ссылки. Обратите внимание, что ссылочные типы, в отличие от большинства других типов, способны иметь спецификаторы `const` как верхнего, так и нижнего уровня, независимо друг от друга.

```
int i = 0;
int *const p1 = &i;           // нельзя изменить значение p1;
                               // const верхнего уровня
const int ci = 42;           // нельзя изменить ci; const верхнего уровня
const int *p2 = &ci;         // нельзя изменить p2; const нижнего уровня
const int *const p3 = p2;    // справа const верхнего уровня, слева нет
const int &r = ci;           // const в ссылочных типах всегда нижнего уровня
```



Различие между спецификаторами `const` верхнего и нижнего уровней проявляется при копировании объекта. При копировании объекта спецификатор `const` верхнего уровня игнорируется.

```
i = ci; // ok: копирование значения ci; спецификатор const верхнего
        // уровня в ci игнорируется
p2 = p3; // ok: указываемые типы совпадают; спецификатор const верхнего
        // уровня в p3 игнорируется
```

Копирование объекта не изменяет копируемый объект. Поэтому несущественно, является ли копируемый или копирующий объект константой.

Спецификатор `const` нижнего уровня, напротив, никогда не игнорируется. При копировании объектов у них обоих должны быть одинаковые спецификаторы `const` нижнего уровня, или должно быть возможно преобразование между типами этих двух объектов. Как правило, преобразование неконстанты в константу возможно, но не наоборот.

```
int *p = p3; // ошибка: p3 имеет const нижнего уровня, а p - нет
p2 = p3;     // ok: p2 имеет то же const нижнего уровня, что и p3
p2 = &i;     // ok: преобразование int* в const int* возможно
int &r = ci;  // ошибка: невозможно связать обычную int& с
               // объектом const int
const int &r2 = i; // ok: const int& можно связать с обычным int
```

У указателя `p3` есть спецификатор `const` нижнего и верхнего уровня. При копировании указателя `p3` можно проигнорировать его спецификатор `const` верхнего уровня, но не тот факт, что он указывает на константный тип. Следовательно, нельзя использовать указатель `p3` для инициализации указателя `p`, который указывает на простой (неконстантный) тип `int`. С другой стороны, вполне можно присвоить указатель `p3` указателю `p2`. У обоих указателей тот же тип (спецификатор `const` нижнего уровня). Тот факт, что `p3` — константный указатель (т.е. у него есть спецификатор `const` верхнего уровня), не имеет значения.

Упражнения раздела 2.4.3

Упражнение 2.30. Укажите по каждому из следующих объявлений, имеет ли объявляемый объект спецификатор `const` нижнего или верхнего уровня.

```
const int v2 = 0;           int v1 = v2;
int *p1 = &v1, &r1 = v1;
const int *p2 = &v2, *const p3 = &i, &r2 = v2;
```

Упражнение 2.31. С учетом объявлений в предыдущем упражнении укажите, допустимы ли следующие присвоения. Объясните, как спецификатор `const` верхнего или нижнего уровня применяется в каждом случае.

```
r1 = v2;
p1 = p2;    p2 = p1;
p1 = p3;    p2 = p3;
```



2.4.4. Переменные `constexpr` и константные выражения

Константное выражение (constant expression) — это выражение, значение которого не может измениться и вычисляется во время компиляции. Литерал — это константное выражение. Константный объект, инициализируемый константным выражением, также является константным выражением. Вскоре мы увидим, что в языке есть несколько контекстов, требующих константных выражений.

Является ли данный объект (или выражение) константным выражением, зависит от типов и инициализаторов. Например:

```
const int max_files = 20;           // max_files - константное выражение
const int limit = max_files + 1;    // limit - константное выражение
int staff_size = 27;                // staff_size - неконстантное выражение
const int sz = get_size();           // sz - неконстантное выражение
```

Хотя переменная `staff_size` инициализируется литералом, это неконстантное выражение, поскольку он имеет обычный тип `int`, а не `const int`. С другой стороны, хоть переменная `sz` и константа, значение ее инициализатора неизвестно до времени выполнения. Следовательно, это неконстантное выражение.

Переменные `constexpr`

В большой системе может быть трудно утверждать (наверняка), что инициализатор — константное выражение. Константная переменная могла бы быть определена с инициализатором, который мы полагаем константным выражением. Однако при использовании этой переменной в контексте, требующем константного выражения, может оказаться, что инициализатор не был константным выражением. Как правило, определение объекта и его использования в таком контексте располагаются довольно далеко друг от друга.

C++
11

Согласно новому стандарту, можно попросить компилятор проверить, является ли переменная константным выражением, используя в ее объявлении ключевое слово `constexpr`. *Переменные* `constexpr` неявно являются константой и должны инициализироваться константными выражениями.

```
constexpr int mf = 20;           // 20 - константное выражение
constexpr int limit = mf + 1;    // mf + 1 - константное выражение
constexpr int sz = size();       // допустимо, только если size() является
                                // функцией constexpr
```

Хоть и нельзя использовать обычную функцию как инициализатор для переменной `constexpr`, как будет описано в разделе 6.5.2 (стр. 312), новый стандарт позволяет определять функции как `constexpr`. Такие функции должны быть достаточно просты, чтобы компилятор мог выполнить их во время компиляции. Функции `constexpr` можно использовать в инициализаторе переменной `constexpr`.



Рекомендуем

Как правило, ключевое слово `constexpr` имеет смысл использовать для переменных, которые предполагается использовать как константные выражения.

Литеральные типы

Поскольку константное выражение обрабатывается во время компиляции, есть пределы для типов, которые можно использовать в объявлении `constexpr`. Типы, которые можно использовать в объявлении `constexpr`, известны как *литеральные типы* (literal type), поскольку они достаточно просты для литеральных значений.

Все использованные до сих пор типы — арифметический, ссылка и указатель — это литеральные типы. Наш класс `Sales_item` и библиотечный тип `string` не относятся к литеральным типам. Следовательно, нельзя определить переменные этих типов как `constexpr`. Другие виды литеральных типов рассматриваются в разделах 7.5.6 (стр. 385) и 19.3 (стр. 1038).

Хотя указатели и ссылки можно определить как `constexpr`, используемые для их инициализации объекты жестко ограничены. Указатель `constexpr` можно инициализировать литералом `nullptr` или литералом (т.е. константным выражением) `0`. Можно также указать на (или связать с) объект, который остается по фиксированному адресу.

По причинам, рассматриваемым в разделе 6.1.1 (стр. 271), определенные в функции переменные обычно не хранятся по фиксированному адресу. Следовательно, нельзя использовать указатель `constexpr` для указания на такие переменные. С другой стороны, адрес объекта, определенного вне любой функции, является константным выражением и, таким образом, может ис-

пользоваться для инициализации указателя `constexpr`. Как будет описано в разделе 6.1.1 (стр. 272), функции могут определять переменные, существующие на протяжении нескольких вызовов этой функции. Как и объект, определенный вне любой функции, эти специальные локальные объекты также имеют фиксированные адреса. Поэтому и ссылка `constexpr` может быть связана с такой переменной, и указатель `constexpr` может содержать ее адрес.

Указатели и спецификатор `constexpr`

Важно понимать, что при определении указателя в объявлении `constexpr` спецификатор `constexpr` относится к указателю, а не к типу, на который указывает указатель.

```
const int *p = nullptr;      // p - указатель на const int
constexpr int *q = nullptr; // q - константный указатель на int
```

Несмотря на внешний вид, типы `p` и `q` весьма различны; `p` — указатель на константу, тогда как `q` — константный указатель. Различие является следствием того факта, что спецификатор `constexpr` налагает на определяемый объект спецификатор `const` верхнего уровня (см. раздел 2.4.3, стр. 101).

Как и любой другой константный указатель, указатель `constexpr` может указать на константный или неконстантный тип.

```
constexpr int *np = nullptr; // np - нулевой константный указатель на int
int j = 0;
constexpr int i = 42;        // типом i является const int
// i и j должны быть определены вне любой функции
constexpr const int *p = &i; // p - константный указатель на const int i
constexpr int *p1 = &j;      // p1 - константный указатель на int j
```

Упражнения раздела 2.4.4

Упражнение 2.32. Допустим ли следующий код? Если нет, то как его исправить?

```
int null = 0, *p = null;
```

2.5. Работа с типами

По мере усложнения программ используемые в них типы также становятся все более сложными. Осложнения в использовании типов возникают по двум причинам. Имена некоторых типов трудно писать по памяти. Написание некоторых их форм утомительно и подвержено ошибкам. Кроме того, формат записи сложного типа способен скрыть его цель или значение. Другой источник осложнений кроется в том, что иногда трудно точно определить необходимый тип. Это может потребовать оглянуться на контекст программы.

2.5.1. Псевдонимы типов

Псевдоним типа (type alias) — это имя, являющееся синонимом имени другого типа. Псевдонимы типа позволяют упростить сложные определения типов, облегчая их использование. Псевдонимы типа позволяют также подчеркивать цель использования типа. Определить псевдоним типа можно одним из двух способов. Традиционно он определяется при помощи ключевого слова `typedef`:

```
typedef double wages;    // wages - синоним для double
typedef wages base, *p;  // base - синоним для double, а p - для double*
```

Ключевое слово `typedef` может быть частью базового типа в объявлении (см. раздел 2.3, стр. 84). Объявления, включающие ключевое слово `typedef`, определяют псевдонимы типа, а не переменные. Как и в любое другое объявление, в это можно включать модификаторы типа, которые определяют составные типы, включающие базовый тип.

 Новый стандарт вводит второй способ определения псевдонима типа при помощи *объявления псевдонима* (alias declaration) и знака `=`.

```
using SI = Sales_item; // SI - синоним для Sales_item
```

Объявление псевдонима задает слева от оператора `=` имя псевдонима типа, который расположен справа.

Псевдоним типа — это имя типа, оно может присутствовать везде, где присутствует имя типа.

```
wages hourly, weekly; // то же, что и double hourly, weekly;
SI item;               // то же, что и Sales_item item
```



Указатели, константы и псевдонимы типа

Объявления, использующие псевдонимы типа, представляющие составные типы и константы, могут приводить к удивительным результатам. Например, следующие объявления используют тип `pstring`, который является псевдонимом для типа `char*`.

```
typedef char *pstring;
const pstring cstr = 0; // cstr - константный указатель на char
const pstring *ps;     // ps - указатель на константный указатель
                       // на тип char
```

Базовым типом в этих объявлениях является `const pstring`. Как обычно, модификатор `const` в базовом типе модифицирует данный тип. Тип `pstring` — это указатель на тип `char`, а `const pstring` — это константный указатель на тип `char`, но не указатель на тип `const char`.

Заманчиво, хоть и неправильно, интерпретировать объявление, которое использует псевдоним типа как концептуальную замену псевдонима, соответствующим ему типом:

```
const char *cstr = 0; // неправильная интерпретация const pstring cstr
```

Однако эта интерпретация неправильна. Когда используется тип `pstring` в объявлении, базовым типом объявления является тип указателя. При перезаписи объявления с использованием `char*`, базовым типом будет `char`, а `*` будет частью оператора объявления. В данном случае базовый тип — это `const char`. Перезапись объявляет `cstr` указателем на тип `const char`, а не константным указателем на тип `char`.



2.5.2. Спецификатор типа `auto`



Нет ничего необычного в желании сохранить значение выражения в переменной. Чтобы объявить переменную, нужно знать тип этого выражения. Когда мы пишем программу, может быть на удивление трудно (а иногда даже невозможно) определить тип выражения. По новому стандарту можно позволить компилятору самому выяснять этот тип. Для этого используется *спецификатор типа* `auto`. В отличие от таких спецификаторов типа, как `double`, задающих определенный тип, спецификатор `auto` приказывает компилятору вывести тип из инициализатора. Само собой разумеется, у переменной, использующей спецификатор типа `auto`, должен быть инициализатор.

```
// тип item выводится из типа результата суммы val1 и val2
auto item = val1 + val2; // item инициализируется результатом val1 + val2
```

Здесь компилятор выведет тип переменной `item` из типа значения, возвращенного при применении оператора `+` к переменным `val1` и `val2`. Если переменные `val1` и `val2` — объекты класса `Sales_item` (см. раздел 1.5, стр. 47), типом переменной `item` будет класс `Sales_item`. Если эти переменные имеют тип `double`, то у переменной `item` будет тип `double` и т.д.

Подобно любому другому спецификатору типа, используя спецификатор `auto`, можно определить несколько переменных. Поскольку объявление может задействовать только один базовый тип, у инициализаторов всех переменных в объявлении должны быть типы, совместимые друг с другом.

```
auto i = 0, *p = &i;    // ok: i - int, a p - указатель на int
auto sz = 0, pi = 3.14; // ошибка: несовместимые типы у sz и pi
```

Составные типы, `const` и `auto`

Выводимый компилятором тип для спецификатора `auto` не всегда точно совпадает с типом инициализатора. Компилятор корректирует тип так, чтобы он соответствовал обычным правилам инициализации.

Во-первых, как уже упоминалось, при использовании ссылки в действительности используется объект, на который она ссылается. В частности, при использовании ссылки как инициализатора им является соответствующий объект. Компилятор использует тип этого объекта для вывода типа `auto`.

```
int i = 0, &r = i;
auto a = r; // a - int (r - псевдоним для i, имеющий тип int)
```

Во-вторых, выводение типа `auto` обычно игнорирует спецификаторы `const` верхнего уровня (см. раздел 2.4.3, стр. 101). Как обычно в инициализациях, спецификаторы `const` нижнего уровня учитываются в случае, когда инициализатор является указателем на константу.

```
const int ci = i, &cr = ci;
auto b = ci; // b - int (const верхнего уровня в ci отброшен)
auto c = cr; // c - int (cr - псевдоним для ci с const верхнего уровня)
auto d = &i; // d - int* (& объекта int - int*)
auto e = &ci; // e - const int* (& константного объекта - const нижнего
// уровня)
```

Если необходимо, чтобы у выведенного типа был спецификатор `const` верхнего уровня, его следует указать явно.

```
const auto f = ci; // выведенный тип ci - int; тип f - const int
```

Можно также указать, что необходима ссылка на автоматически выведенный тип. Обычные правила инициализации все еще применимы.

```
auto &g = ci; // g - const int&, связанный с ci
auto &h = 42; // ошибка: нельзя связать простую ссылку с литералом
const auto &j = 42; // ok: константную ссылку с литералом связать можно
```

Когда запрашивается ссылка на автоматически выведенный тип, спецификаторы `const` верхнего уровня в инициализаторе не игнорируются. Как обычно при связывании ссылки с инициализатором, спецификаторы `const` не относятся к верхнему уровню.

При определении нескольких переменных в том же операторе важно не забывать, что ссылка или указатель — это часть специфического оператора объявления, а не часть базового типа объявления. Как обычно, инициализаторы должны быть совместимы с автоматически выведенными типами:

```
auto k = ci, &l = i; // k - int; l - int&
auto &m = ci, *p = &ci; // m - const int&; p - указатель на const int
// ошибка: выводение типа из i - int; тип, выведенный из &ci - const int
auto &n = i, *p2 = &ci;
```

Упражнения раздела 2.5.2

Упражнение 2.33. С учетом определения переменных из этого раздела укажите то, что происходит в каждом из этих присвоений.

```
a = 42;    b = 42;    c = 42;
d = 42;    e = 42;    g = 42;
```

Упражнение 2.34. Напишите программу, содержащую переменные и присвоения из предыдущего упражнения. Выведите значения переменных до и после присвоений, чтобы проверить правильность предположений в предыдущем упражнении. Если они неправильны, изучите примеры еще раз и выясните, что привело к неправильному заключению.

Упражнение 2.35. Укажите типы, выведенные в каждом из следующих определений. Затем напишите программу, чтобы убедиться в своей правоте.

```
const int i = 42;
auto j = i; const auto &k = i; auto *p = &i;
const auto j2 = i, &k2 = i;
```



2.5.3. Спецификатор типа `decltype`



Иногда необходимо определить переменную, тип которой компилятор выводит из выражения, но не использовать это выражение для инициализации переменной. Для таких случаев новый стандарт вводит спецификатор типа `decltype`, возвращающий тип его операнда. Компилятор анализирует выражение и определяет его тип, но не вычисляет его результат.

```
decltype(f()) sum = x; // sum имеет тот тип, который возвращает функция f
```

Здесь компилятор не вызывает функцию `f()`, но он использует тип, который возвратил бы такой вызов для переменной `sum`. Таким образом, компилятор назначает переменной `sum` тот же тип, который был бы возвращен при вызове функции `f()`.

Таким образом, спецификатор `decltype` учитывает спецификатор `const` верхнего уровня и ссылки, но несколько отличается от того, как работает спецификатор `auto`. Когда выражение, к которому применен спецификатор `decltype`, является переменной, он возвращает тип этой переменной, включая спецификатор `const` верхнего уровня и ссылки.

```
const int ci = 0, &cj = ci;
decltype(ci) x = 0; // x имеет тип const int
decltype(cj) y = x; // y имеет тип const int& и связана с x
decltype(cj) z; // ошибка: z - ссылка, она должна быть инициализирована
```

Поскольку `cj` — ссылка, `decltype(cj)` — ссылочный тип. Как и любую другую ссылку, ссылку `z` следует инициализировать.

Следует заметить, что спецификатор `decltype` — единственный контекст, в котором переменная определена, поскольку ссылка не рассматривается как синоним объекта, на который она ссылается.



Спецификатор `decltype` и ссылки

Когда спецификатор `decltype` применяется к выражению, которое не является переменной, получаемый тип соответствует типу выражения. Как будет продемонстрировано в разделе 4.1.1 (стр. 189), некоторые выражения заставят спецификатор `decltype` возвращать ссылочный тип. По правде говоря, спецификатор `decltype` возвращает ссылочный тип для выражений, результатом которых являются объекты, способные стоять слева от оператора присвоения.

```
// decltype выражение может быть ссылочным типом
int i = 42, *p = &i, &r = i;
decltype(r + 0) b; // ok: сложение возвращает тип int; b имеет тип int
                  // (не инициализирована)
decltype(*p) c;    // ошибка: c имеет тип int& и требует инициализации
```

Здесь `r` — ссылка, поэтому `decltype(r)` возвращает ссылочный тип. Если необходим тип, на который ссылается ссылка `r`, можно использовать ее в таком выражении, как `r + 0`, поскольку оно возвращает значение не ссылочного типа.

С другой стороны, оператор обращения к значению — пример выражения, для которого спецификатор `decltype` возвращает ссылку. Как уже упоминалось, при обращении к значению указателя возвращается объект, на который он указывает. Кроме того, этому объекту можно присвоить значение. Таким образом, `decltype(*p)` выведет тип `int&`, а не просто `int`.



Еще одно важное различие между спецификаторами `decltype` и `auto` в том, что выводение, осуществляемое спецификатором `decltype`, *зависит от формы данного выражения*. Не всегда понимают то, что включение имени переменной в круглые скобки влияет на тип, возвращаемый спецификатором `decltype`. При применении спецификатора `decltype` к переменной без круглых скобок получается тип этой переменной. Если заключить имя переменной в одни или несколько круглых скобок, то компилятор будет рассматривать операнд как выражение. Переменная — это выражение, которое способно быть левым операндом присвоения. В результате спецификатор `decltype` для такого выражения возвратит ссылку.

```
// decltype переменной в скобках - всегда ссылка
decltype((i)) d; // ошибка: d - int& и должна инициализироваться
decltype(i) e;   // ok: e имеет тип int (не инициализирована)
```




Помните, что спецификатор `decltype((переменная))` (обратите внимание на парные круглые скобки) *всегда* возвращает ссылочный тип, а спецификатор `decltype(переменная)` возвращает ссылочный тип, только если *переменная* является ссылкой.

Упражнения раздела 2.5.3

Упражнение 2.36. Определите в следующем коде тип каждой переменной и значения, которые будет иметь каждая из них по завершении.

```
int a = 3, b = 4;
decltype(a) c = a;
decltype((b)) d = a;
++c;
++d;
```

Упражнение 2.37. Присвоение — это пример выражения, которое возвращает ссылочный тип. Тип — это ссылка на тип левого операнда. Таким образом, если переменная `i` имеет тип `int`, то выражение `i = x` имеет тип `int&`. С учетом этого определите тип и значение каждой переменной в следующем коде:

```
int a = 3, b = 4;
decltype(a) c = a;
decltype(a = b) d = a;
```

Упражнение 2.38. Опишите различия вывода типа спецификаторами `decltype` и `auto`. Приведите пример выражения, где спецификаторы `auto` и `decltype` выведут тот же тип, и пример, где они выведут разные типы.



2.6. Определение собственных структур данных

На самом простом уровне *структура данных* (data structure) — это способ группировки взаимосвязанных данных и стратегии их использования. Например, класс `Sales_item` группирует ISBN книги, количество проданных экземпляров и выручку от этой продажи. Он предоставляет также набор операций, таких как функция `isbn()` и операторы `>>`, `<<`, `+` и `+=`.

В языке C++ мы создаем собственные типы данных, определяя класс. Такие библиотечные типы, как `string`, `istream` и `ostream`, определены как классы, подобно типу `Sales_item` в главе 1. Поддержка классов в языке C++ весьма обширна, фактически части III и IV в значительной степени посвящены описанию средств, связанных с классами. Хотя класс `Sales_item` довольно прост, мы не сможем определить его полностью, пока не узнаем в главе 14, как писать собственные операторы.



2.6.1. Определение типа `Sales_data`

Несмотря на то что мы еще не можем написать свой класс `Sales_item` полностью, уже вполне можно создать достаточно реалистичный класс, группирующий необходимые элементы данных. Стратегия использования этого класса заключается в том, что пользователи будут получать доступ непосредственно к элементам данных и смогут самостоятельно реализовать необходимые операции.

Поскольку создаваемая структура данных не поддерживает операций, назовем новую версию `Sales_data`, чтобы отличать ее от типа `Sales_item`. Определим класс следующим образом:

```
struct Sales_data {
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
```

Класс начинается с ключевого слова `struct`, сопровождаемого именем класса и (возможно пустым) телом класса. Тело класса заключено в фигурные скобки и формирует новую область видимости (см. раздел 2.2.4, стр. 82). Определенные в классе имена должны быть уникальны в пределах класса, но вне класса они могут повторяться.

Ближайшие фигурные скобки, заключающие тело класса, следует сопровождать точкой с запятой. Точка с запятой необходима, так как после тела класса можно определить переменные:

```
struct Sales_data { /* ... */ } accum, trans, *salesptr;
// эквивалентно, но лучше определять эти объекты так
struct Sales_data { /* ... */ };
Sales_data accum, trans, *salesptr;
```

Точка с запятой отмечает конец (обычно пустого) списка объявления операторов. Обычно определение объекта в составе определения класса — это не лучшая идея. Объединение в одном операторе определений двух разных сущностей (класса и переменной) ухудшает читабельность кода.



ВНИМАНИЕ

Забывтая точка с запятой в конце определения класса — довольно распространенная ошибка начинающих программистов.

Переменные-члены класса

В теле класса определены *члены* (member) класса. У нашего класса есть только *переменные-члены* (data member). Переменные-члены класса определяют содержимое объектов этого класса. Каждый объект обладает собственным экземпляром переменных-членов класса. Изменение переменных-членов одного объекта не изменяет данные в любом другом объекте класса `Sales_data`.

Переменные-члены определяют точно так же, как и обычные переменные: указывается базовый тип, затем список из одного или нескольких операторов объявления. У нашего класса будут три переменные-члены: член типа `string` по имени `bookNo`, член типа `unsigned` по имени `units_sold` и член типа `double` по имени `revenue`. Эти три переменные-члены будут у каждого объекта класса `Sales_data`.

**C++
11** По новому стандарту переменной-члену можно предоставить *внутри-классовый инициализатор* (in-class initializer). Он используется для инициализации переменных-членов при создании объектов. Члены без инициализатора инициализируются по умолчанию (см. раздел 2.2.1, стр. 77). Таким образом, при определении объектов класса `Sales_data` переменные-члены `units_sold` и `revenue` будут инициализированы значением 0, а переменная-член `bookNo` — пустой строкой.

Внутриклассовые инициализаторы ограничены формой их использования (см. раздел 2.2.1, стр. 77): они должны либо быть заключены в фигурные скобки, либо следовать за знаком `=`. Нельзя определить внутриклассовый инициализатор в круглых скобках.

В разделе 7.2 (стр. 350) указано, что язык C++ обладает еще одним ключевым словом, `class`, также используемым для определения собственной структуры данных. В этом разделе используем ключевое слово `struct`, поскольку пока еще не рассмотрены приведенные в главе 7 дополнительные средства, связанные с классом.

Упражнения раздела 2.6.1

Упражнение 2.39. Откомпилируйте следующую программу и посмотрите, что будет, если не поставить точку с запятой после определения класса. Запомните полученное сообщение, чтобы узнать его в будущем.

```
struct Foo { /* пусто */ } // Примечание: нет точки с запятой
int main()
{
    return 0;
}
```

Упражнение 2.40. Напишите собственную версию класса `Sales_data`.

2.6.2. Использование класса `Sales_data`

В отличие от класса `Sales_item`, класс `Sales_data` не поддерживает операций. Пользователи класса `Sales_data` должны сами писать все операции, в которых они нуждаются. В качестве примера напомним новую версию программы из раздела 1.5.2 (стр. 51), которая выводила сумму двух транзакций. Программа будет получать на входе такие транзакции:

```
0-201-78345-X 3 20.00
0-201-78345-X 2 25.00
```

Каждая транзакция содержит ISBN, количество проданных книг и цену, по которой была продана каждая книга.

Суммирование двух объектов класса `Sales_data`

Поскольку класс `Sales_data` не предоставляет операций, придется написать собственный код, осуществляющий ввод, вывод и сложение. Будем подразумевать, что класс `Sales_data` определен в заголовке `Sales_data.h`. Определение заголовка рассмотрим в разделе 2.6.3 (стр. 116).

Так как эта программа будет длиннее любой, написанной до сих пор, рассмотрим ее по частям. В целом у программы будет следующая структура:

```
#include <iostream>
#include <string>
#include "Sales_data.h"
int main()
{
    Sales_data data1, data2;
    // код чтения данных в data1 и data2
    // код проверки наличия у data1 и data2 одинакового ISBN
    // если это так, то вывести сумму data1 и data2
}
```

Как и первоначальная программа, эта начинается с включения заголовков, необходимых для определения переменных, содержащих ввод. Обратите внимание, что, в отличие от версии `Sales_item`, новая программа включает заголовок `string`. Он необходим потому, что код должен манипулировать переменной-членом `bookNo` типа `string`.

Чтение данных в объект класса `Sales_data`

Хотя до глав 3 и 10 мы не будем описывать библиотечный тип `string` подробно, упомянем пока лишь то, что необходимо знать для определения и использования члена класса, содержащего ISBN. Тип `string` содержит последовательность символов. Он имеет операторы `>>`, `<<` и `==` для чтения, записи и сравнения строк соответственно. Этих знаний достаточно для написания кода чтения первой транзакции.

```
double price = 0; // цена за книгу, используемая для вычисления
                  // общей выручки
// читать первую транзакцию: ISBN, количество проданных книг, цена книги
std::cin >> data1.bookNo >> data1.units_sold >> price;
// вычислить общий доход из price и units_sold
data1.revenue = data1.units_sold * price;
```

Транзакции содержат цену, по которой была продана каждая книга, но структура данных хранит общий доход. Данные транзакции будем читать

в переменную `price` (цена) типа `double`, исходя из которой и вычислим член `revenue` (доход).

```
std::cin >> data1.bookNo >> data1.units_sold >> price;
```

Для чтения значений членов `bookNo` и `units_sold` (продано экземпляров) объекта по имени `data1` оператор ввода использует точечный оператор (см. раздел 1.5.2, стр. 51).

Последний оператор присваивает произведение `data1.units_sold` и `price` переменной-члену `revenue` объекта `data1`.

Затем программа повторяет тот же код для чтения данных в объект `data2`.

```
// читать вторую транзакцию
std::cin >> data2.bookNo >> data2.units_sold >> price;
data2.revenue = data2.units_sold * price;
```

Вывод суммы двух объектов класса `Sales_data`

Следующая задача — проверить наличие у транзакций одинакового ISBN. Если это так, вывести их сумму, в противном случае отобразить сообщение об ошибке.

```
if (data1.bookNo == data2.bookNo) {
    unsigned totalCnt = data1.units_sold + data2.units_sold;
    double totalRevenue = data1.revenue + data2.revenue;
    // вывести: ISBN, общее количество проданных экземпляров,
    // общий доход, среднюю цену за книгу
    std::cout << data1.bookNo << " " << totalCnt
              << " " << totalRevenue << " ";
    if (totalCnt != 0)
        std::cout << totalRevenue/totalCnt << std::endl;
    else
        std::cout << "(no sales)" << std::endl;
    return 0; // означает успех
} else { // транзакции не для того же ISBN
    std::cerr << "Data must refer to the same ISBN"
              << std::endl;
    return -1; // означает неудачу
}
```

Первый оператор `if` сравнивает члены `bookNo` объектов `data1` и `data2`. Если эти члены содержат одинаковый ISBN, выполняется код в фигурных скобках, суммирующий компоненты двух переменных. Поскольку необходимо вывести среднюю цену, сначала вычислим общее количество проданных экземпляров и общий доход, а затем сохраним их в переменных `totalCnt` и `totalRevenue` соответственно. Выводим эти значения, а затем проверяем, были ли книги проданы, и если да, то выводим вычисленную среднюю цену за книгу. Если никаких продаж не было, выводим сообщение, обращающее внимание на этот факт.

Упражнения раздела 2.6.2

Упражнение 2.41. Используйте класс `Sales_data` для перезаписи кода упражнений из разделов 1.5.1 (стр. 50), 1.5.2 (стр. 52) и 1.6 (стр. 54). А также определите свой класс `Sales_data` в том же файле, что и функция `main()`.



2.6.3. Создание собственных файлов заголовка

Как будет продемонстрировано в разделе 19.7 (стр. 1063), класс можно определить в функции, однако такие классы ограничены по функциональным возможностям. Поэтому классы обычно не определяют в функциях. При определении класса за пределами функции в каждом файле исходного кода может быть только одно определение этого класса. Кроме того, если класс используется в нескольких разных файлах, определение класса в каждом файле должно быть тем же.

Чтобы гарантировать совпадение определений класса в каждом файле, классы обычно определяют в файлах заголовка. Как правило, классы хранятся в заголовках, имя которых совпадает с именем класса. Например, библиотечный тип `string` определен в заголовке `string`. Точно так же, как уже было продемонстрировано, наш класс `Sales_data` определен в файле заголовка `Sales_data.h`.

Заголовки (обычно) содержат сущности (такие как определения класса или переменных `const` и `constexpr` (см. раздел 2.4 (стр. 96)), которые могут быть определены в любом файле только однажды. Однако заголовки нередко должны использовать средства из других заголовков. Например, поскольку у класса `Sales_data` есть член типа `string`, заголовок `Sales_data.h` должен включать заголовок `string`. Как уже упоминалось, программы, использующие класс `Sales_data`, должны также включать заголовок `string`, чтобы использовать член `bookNo`. В результате использующие класс `Sales_data` программы будут включать заголовок `string` дважды: один раз непосредственно и один раз как следствие включения заголовка `Sales_data.h`. Поскольку заголовок мог бы быть включен несколько раз, код необходимо писать так, чтобы обезопасить от многократного включения.



После внесения любых изменений в заголовки необходимо перекомпилировать все использующие его файлы исходного кода, чтобы вступили в силу новые или измененные объявления.

Краткое введение в препроцессор

Наиболее распространенный способ обезопасить заголовки от многократного включения подразумевает использование препроцессора. *Препроцессор*

(preprocessor), унаследованный языком C++ от языка C, является программой, которая запускается перед компилятором и изменяет исходный текст программ. Наши программы уже полагаются на такое средство препроцессора, как директива `#include`. Когда препроцессор встречает директиву `#include`, он заменяет ее содержимым указанного заголовка.

Программы C++ используют также препроцессор для *защиты заголовка* (header guard). Защита заголовка полагается на переменные препроцессора (см. раздел 2.3.2, стр. 89). Переменные препроцессора способны находиться в одном из двух состояний: она либо определена, либо не определена. Директива `#define` получает имя и определяет его как переменную препроцессора. Есть еще две директивы, способные проверить, определена ли данная переменная препроцессора или нет. Директива `#ifdef` истинна, если переменная была определена, а директива `#ifndef` истинна, если переменная *не была* определена. В случае истинности проверки выполняется все, что расположено после директивы `#ifdef` или `#ifndef` и до соответствующей директивы `#endif`.

Эти средства можно использовать для принятия мер против множественного включения следующим образом:

```
#ifndef SALES_DATA_H
#define SALES_DATA_H
#include <string>
struct Sales_data {
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
#endif
```

При первом включении заголовка `Sales_data.h` директива `#ifndef` истинна, и препроцессор обрабатывает строки после нее до директивы `#endif`. В результате переменная препроцессора `SALES_DATA_H` будет определена, а содержимое заголовка `Sales_data.h` скопировано в программу. Если впоследствии включить заголовок `Sales_data.h` в тот же файл, то директива `#ifndef` окажется ложна и строки между ней и директивой `#endif` будут проигнорированы.



ВНИМАНИЕ

Имена переменных препроцессора не подчиняются правилам областей видимости языка C++.

Переменные препроцессора, включая имена для защиты заголовка, должны быть уникальными во всей программе. Обычно мы гарантируем уникальность имен защиты заголовка, включая в него имя класса. Чтобы избежать конфликта имен с другими сущностями программы, имена переменных препроцессора обычно пишут полностью в верхнем регистре.

Рекомендуем

У заголовков должна быть защита, даже если они не включаются в другие заголовки. Защита заголовка проста в написании, и при привычном их определении не нужно размышлять, нужны они или нет.

Упражнения раздела 2.6.3

Упражнение 2.42. Напишите собственную версию заголовка `Sales_data.h` и используйте его для новой версии упражнения из раздела 2.6.2 (стр. 116).

Резюме

Типы — фундаментальная часть всех программ C++.

Каждый тип определяет требования по хранению и операциям, которые можно выполнять с объектами этого типа. Язык предоставляет набор фундаментальных встроенных типов, таких как `int` и `char`, которые тесно связаны с их представлением на аппаратных средствах машины. Типы могут быть неконстантными или константными; константный объект следует инициализировать. Будучи однажды инициализированным, значение константного объекта не может быть изменено. Кроме того, можно определить составные типы, такие как указатели или ссылки. Составной тип — это тип, определенный в терминах другого типа.

Язык позволяет определять собственные типы, т.е. классы. Библиотека использует классы, чтобы предоставить набор таких высокоуровневых абстракций, как типы `IO` и `string`.

Термины

Адрес (address). Номер байта в памяти, начиная с которого располагается объект.

Арифметический тип (arithmetic type). Встроенные типы, представляющие логические значения, символы, целые числа и числа с плавающей запятой.

Базовый тип (base type). Спецификатор типа, возможно со спецификатором `const`, который предшествует оператору объявления в объявлении. Базовый тип представляет общий тип, на основании которого строятся операторы объявления в объявлении.

Байт (byte). Наименьший адресуемый блок памяти. На большинстве машин байт составляет 8 битов.

Беззнаковый тип (unsigned). Целочисленный тип данных, переменные которого способны хранить значения больше или равные нулю.

В области видимости (in scope). Имя, которое видимо от текущей области видимости.

Внешняя область видимости (outer scope). Область видимости, включающая другую область видимости.

Внутренняя область видимости (inner scope). Область видимости, вложенная в другую область видимости.

Внутриклассовый инициализатор (in-class initializer). Инициализатор, предоставленный как часть объявления переменной-члена класса. За внутриклассовым инициализатором следует символ =, или он заключается в фигурные скобки.

Временный объект (temporary). Безымянный объект, создаваемый компилятором при вычислении выражения. Временный объект существует до конца вычисления всего выражения, для которого он был создан.

Глобальная область видимости (global scope). Область видимости, внешняя для всех остальных областей видимости.

Директива препроцессора #define. Определяет переменную препроцессора.

Директива препроцессора #endif. Завершает область #ifdef или #ifndef.

Директива препроцессора #ifdef. Выясняет, что данная переменная определена.

Директива препроцессора #ifndef. Выясняет, что данная переменная не определена.

Защита заголовка (header guard). Переменная препроцессора, предназначенная для предотвращения неоднократного подключения содержимого заголовка в один файл исходного кода.

Знаковый тип (signed). Целочисленный тип данных, переменные которого способны хранить отрицательные и положительные числа, включая ноль.

Идентификатор (identifier). Последовательность символов, составляющая имя. Идентификатор зависит от регистра символов.

Инициализация (initialization). Присвоение переменной исходного значения при ее определении. Обычно переменные следует инициализировать.

Инициализация по умолчанию (default initialization). Способ инициализации объектов при отсутствии явной инициализации. Инициализация объектов типа класса определяется классом. Объекты встроенного типа, определенного в глобальной области видимости, инициализируются значением 0, а определенные в локальной области видимости остаются неинициализированными и имеют неопределенное значение.

Интегральный тип (integral type). То же, что и арифметический или целочисленный тип.

Ключевое слово struct. Используется при определении структуры (класса).

Ключевое слово typedef. Позволяет определить псевдоним для другого типа. Когда ключевое слово typedef присутствует в объявлении базового типа, определенные в объявлении имена становятся именами типа.

Константная ссылка (const reference). Разговорный термин для ссылки на константный объект.

Константное выражение (constant expression). Выражение, значение которого может быть вычислено во время компиляции.

Константный указатель (const pointer). Указатель со спецификатором const.

Контроль соответствия типов (type checking). Термин, описывающий процесс проверки компилятором соответствия способа использования объекта заявленному для него типу.

Литерал (literal) Значение, такое как число, символ или строка символов. Это значение не может быть изменено. Символьные литералы заключают в одинарные кавычки, а строковые литералы в двойные.

Литерал `nullptr`. Литеральная константа, означающая нулевой указатель.

Локальная область видимости (local scope). Разговорный синоним для области действия блока кода.

Массив (array). Структура данных, содержащая коллекцию неименованных объектов, к которым можно обращаться по индексу. Более подробная информация о массивах приведена в разделе 3.5.

Неинициализированная переменная (uninitialized variable). Переменная, определенная без исходного значения. Обычно попытка доступа к значению неинициализированной переменной приводит к неопределенному поведению.

Неопределенное поведение (undefined behavior). Случай, для которого стандарт языка не определяет значения. Осознанно или неосознанно, но полагаться на неопределенное поведение нельзя. Оно является источником трудно обнаруживаемых ошибок времени выполнения, проблем безопасности и переносимости.

Непечатаемый символ (nonprintable character). Символ, не имеющий видимого представления, например символ возврата на один символ, символ новой строки и т.д.

Нулевой указатель (null pointer). Указатель со значением 0. Нулевой указатель допустим, но не указывает ни на какой объект.

Область видимости (scope). Часть программы, в которой имена имеют смысл. Язык C++ имеет несколько уровней областей видимости.

Глобальная (global) — имена, определенные вне остальных областей видимости.

Класса (class) — имена, определенные классом.

Пространства имен (namespace) — имена, определенные в пространстве имен.

Блока (block) — имена, определенные в блоке операторов, т.е. в паре фигурных скобок.

Области видимости могут быть вложенными. Как только имя объявлено, оно доступно до конца той области видимости, в которой было объявлено.

Объект (object). Область памяти, которая имеет тип. Переменная — это объект, который имеет имя.

Объявление (declaration). Уведомление о существовании переменной, функции или типа, определяемых в другом месте программы. Никакие имена не могут быть использованы, пока они не определены или не объявлены.

Объявление псевдонима (alias declaration). Определяет синоним для другого типа. Объявление в формате `using имя = тип` объявляет *имя* как синоним типа *тип*.

Оператор &. Оператор обращения к адресу. Возвращает адрес объекта, к которому он был применен.

Оператор *. Оператор обращения к значению. Обращение к значению указателя возвращает объект, на который указывает указатель. Присвоение результату оператора обращения к значению присваивает новое значение основному объекту.

Оператор объявления (declarator). Часть объявления, включающая определяемое имя и, необязательно, модификатор типа.

Определение (definition). Резервирует область в памяти для хранения данных переменной и (необязательно) инициализирует ее значение. Никакие имена не могут быть использованы, пока они не определены или не объявлены.

Переменная (variable). Именованный объект или ссылка. В языке C++ переменные должны быть объявлены перед использованием.

- Переменная** `constexpr`. Переменная, которая представляет константное выражение. Функции `constexpr` рассматриваются в разделе 6.5.2 (стр. 312).
- Переменная препроцессора** (`preprocessor variable`). Переменная, используемая препроцессором. Препроцессор заменяет каждую переменную препроцессора ее значением прежде, чем программа будет откомпилирована.
- Переменная-член** (`data member`). Элемент данных, которые составляют объект. Каждый объект некоего класса обладает собственными экземплярами переменных-членов. Переменные-члены могут быть инициализированы в объявлении класса.
- Преобразование** (`conversion`). Процесс, в результате которого значение одного типа преобразуется в значение другого. Преобразования между встроенными типами определены в самом языке.
- Препроцессор** (`preprocessor`). Препроцессор — это программа, автоматически запускаемая перед компилятором C++.
- Псевдоним типа** (`type alias`). Имя, являющееся синонимом для другого типа. Определяется при помощи ключевого слова `typedef` или объявления псевдонима.
- Раздельная компиляция** (`separate compilation`). Возможность разделить программу на несколько отдельных файлов исходного кода.
- Связывание** (`bind`). Соединение имени с указанной сущностью, чтобы использование имени приводило к использованию основной сущности. Например, ссылка — это имя, связанное с объектом.
- Слово** (`word`). Специфический для каждой машины размер блока памяти, применяемый при целочисленных вычислениях. Обычно размер слова достаточно велик, чтобы содержать адрес. 32-битовое слово обычно занимает 4 байта.
- Составной тип** (`compound type`). Тип, определенный в терминах другого типа.
- Спецификатор** `auto`. Спецификатор типа, позволяющий вывести тип переменной из ее инициализатора.
- Спецификатор** `const` **верхнего уровня** (`top-level const`). Спецификатор `const`, указывающий, что объект не может быть изменен.
- Спецификатор** `const` **нижнего уровня** (`low-level const`). Спецификатор `const` не верхнего уровня. Такие спецификаторы `const` являются неотъемлемой частью типа и никогда не игнорируются.
- Спецификатор** `const`. Спецификатор типа, определяющий объекты, которые не могут быть изменены. Константные объекты следует инициализировать, поскольку нет никакого способа присвоить им значение после определения.
- Спецификатор** `decltype`. Спецификатор типа, позволяющий вывести тип переменной или выражения.
- Спецификатор типа** (`type specifier`). Имя типа.
- Списочная инициализация** (`list initialization`). Форма инициализации, подразумевающая использование фигурных скобок для включения одного или нескольких инициализаторов.
- Ссылка** (`reference`). Псевдоним другого объекта.
- Ссылка на константу** (`reference to const`). Ссылка, неспособная изменить значение объекта, на который она ссылается. Ссылка на константу может быть связана с константным, не-константным объектом или с результатом выражения.

Тип `string`. Библиотечный тип, представляющий последовательность символов переменной длины.

Тип `void*`. Специальный тип указателя, способного указывать на любой неконстантный тип. Обращение к значению таких указателей невозможно.

Тип `void`. Специальный тип без значения и допустимых операций. Нельзя определить переменную типа `void`.

Указатель (pointer). Объект, способный содержать адрес объекта, следующий адрес за концом объекта или нуль.

Указатель на константу (pointer to const). Указатель, способный содержать адрес константного объекта. Указатель на константу не может использоваться для изменения значения объекта, на который он указывает.

Управляющая последовательность (escape sequence). Альтернативный механизм представления символов. Обычно используется для представления непечатаемых символов, таких как символ новой строки или табуляции. Управляющая последовательность состоит из символа наклонной черты влево, сопровождаемой символом, восьмеричным числом из трех цифр, или символа `x`, сопровождаемого шестнадцатеричным числом.

Член класса (class member, member). Часть класса.